IF3270 Pembelajaran Mesin

Feedforward Neural Network

Laporan Tugas Besar 1

Disusun untuk memenuhi tugas mata kuliah Pembelajaran Mesin pada Semester 6 (enam)

Tahun Akademik 2025/2026

Disusun Oleh:

Ahmad Syafiq	**(13523135)**
Muhammad Kinan Arkansyaddad	**(13523152)**
Fachriza Ahmad Setiyono	**(13523162)**

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG

2026

DAFTAR ISI

DAFTAR ISI **2**
BAB I
DESKRIPSI PERSOALAN **4**
Informasi Dataset **8**
BAB II
PEMBAHASAN **10**
2.1. Kelas 10
2.1.1. Tensor 10
2.1.2. Module 12
2.1.3. Layer 12
2.1.4. FFNN 14
2.1.5. BaseActivation 16
2.1.6. Linear 16
2.1.7. ReLU 16
2.1.8. Sigmoid 17
2.1.9. TanH 17
2.1.10. Softmax 18
2.1.11. LeakyReLU 19
2.1.12. BaseLoss 20
2.1.13. MSELoss 20
2.1.14. BinaryClassEntropy 20
2.1.15. CategoricalClassEntropy 21
2.1.16. BaseOptimizer 21
2.1.17. SGD 22
2.2. Hasil Eksperimen dan Analisis 22
2.2.1. Pengaruh Depth dan Width Layer 22
2.2.2. Pengaruh Fungsi Aktivasi 34
2.2.3. Pengaruh Learning Rate 41
2.2.4. Pengaruh Metode Inisialisasi Bobot 46
2.2.5. Pengaruh Regularisasi 52
2.2.6. Perbandingan dengan Pustaka scikit-learn 56
2.3. Analisis 57
BAB III
KESIMPULAN & SARAN **58**
3.1. Kesimpulan 58
3.2. Saran 58
REFERENSI **59**
LAMPIRAN **60**
Link Repositori 60
Tabel Pembagian Tugas 60

Form Penggunaan AI..... 61
Kode Sumber..... 63
engine.py..... 63
nn.py..... 65
activations.py..... 73
loss.py..... 76
optimizers.py..... 77

BAB I
DESKRIPSI PERSOALAN

Implementasikan suatu modul FFNN yang memenuhi ketentuan-ketentuan berikut:

- FFNN yang diimplementasikan dapat **menerima jumlah neuron dari tiap layer** (termasuk input layer dan output layer).
- FFNN yang diimplementasikan dapat **menerima fungsi aktivasi dari tiap layer**. Pilihan fungsi aktivasi yang harus diimplementasikan adalah sebagai berikut:

Nama Fungsi Aktivasi	Definisi Fungsi
Linear	$Linear(x) = x$
ReLU	$ReLU(x) = \max(0, x)$
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$
Hyperbolic Tangent (tanh)	$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$
Softmax	Untuk vector $\vec{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, $softmax(\vec{x})_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$

- FFNN yang diimplementasikan dapat **menerima fungsi loss** dari model tersebut. Pilihan loss function yang harus diimplementasikan adalah sebagai berikut:

Nama Fungsi Aktivasi	Definisi Fungsi
MSE	$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

Binary Cross-Entropy	$\mathcal{L}_{BCE} = -\frac{1}{n}\sum_{i=1}^{n}(y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i))$ y_i = Actual binary label (0 or 1) $\hat{y}_i$ = Predicted value of y_i n = Batch size
Categorical Cross-Entropy	$\mathcal{L}_{CCE} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}(y_{ij} \log \hat{y}_{ij})$ y_{ij} = Actual value of instance i for class j $\hat{y}_{ij}$ = Predicted value of y_{ij} C = Number of classes n = Batch size

    - Catatan:
        - Binary cross-entropy merupakan kasus khusus categorical cross-entropy dengan kelas sebanyak 2.
        - Log yang digunakan merupakan logaritma natural (logaritma dengan basis e).
- Terdapat mekanisme untuk **inisialisasi bobot** tiap neuron (termasuk bias). Pilihan metode inisialisasi bobot yang harus diimplementasikan adalah sebagai berikut:
    - **Zero initialization**
    - Random dengan distribusi **uniform**.
        - Menerima parameter lower bound (batas minimal) dan upper bound (batas maksimal).
        - Menerima parameter seed untuk reproducibility.
    - Random dengan distribusi **normal**.
        - Menerima parameter mean dan variance.
        - Menerima parameter seed untuk reproducibility.
- Instance model yang diinisialisasikan harus bisa **menyimpan bobot** tiap neuron (termasuk bias).
- Instance model yang diinisialisasikan harus bisa **menyimpan gradien bobot** tiap neuron (termasuk bias).
- Instance model memiliki method untuk **menampilkan distribusi bobot** dari tiap layer.
    - Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot.
- Instance model memiliki method untuk **menampilkan distribusi gradien bobot** dari tiap layer.
    - Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot.
- Instance model memiliki method untuk ***save*** dan ***load***.
- Model memiliki implementasi **forward propagation** dengan ketentuan sebagai berikut:

- Dapat menerima input berupa **batch**.

- Model memiliki implementasi **backward propagation** untuk menghitung perubahan gradien:
  - Dapat menangani perhitungan perubahan gradien untuk input data **batch**.
  - Gunakan konsep **chain rule** untuk menghitung gradien tiap bobot terhadap loss function.
  - Berikut merupakan **turunan pertama** untuk setiap fungsi aktivasi:

Nama Fungsi Aktivasi	Turunan Pertama
Linear	$$\frac{d\big(Linear(x)\big)}{dx} = 1$$
ReLU	$$\frac{d\big(ReLU(x)\big)}{dx} = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$$
Sigmoid	$$\frac{d\big(\sigma(x)\big)}{dx} = \sigma(x)\big(1 - \sigma(x)\big)$$
Hyperbolic Tangent (tanh)	$$\frac{d(\tanh(x))}{dx} = \left(\frac{2}{e^{x} - e^{-x}}\right)^2$$
Softmax	Untuk vector $\vec{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$, $$\frac{d(softmax(\vec{x})_i)}{d\vec{x}} = \begin{bmatrix} \frac{\partial(softmax(\vec{x})_1)}{\partial x_1} & \cdots & \frac{\partial(softmax(\vec{x})_1)}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial(softmax(\vec{x})_n)}{\partial x_1} & \cdots & \frac{\partial(softmax(\vec{x})_n)}{\partial x_n} \end{bmatrix}$$ Dimana untuk $i, j \in \{1, \dots, n\}$, $$\frac{\partial(softmax(\vec{x})_i)}{\partial x_j} = softmax(\vec{x})_i\big(\delta_{i,j} - softmax(\vec{x})_j\big)$$ $$\delta_{i,j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

  - Berikut merupakan **turunan pertama** untuk setiap fungsi loss terhadap bobot suatu FFNN (lanjutkan sisanya menggunakan chain rule):

Nama Fungsi Loss	Definisi Fungsi
MSE	$$\frac{\partial \mathcal{L}_{MSE}}{\partial W} = -\frac{2}{n}\sum_{i=1}^{n}\frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}_i}\frac{\partial \hat{y}_i}{\partial W} = -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)\frac{\partial \hat{y}_i}{\partial W}$$
Binary Cross-Entropy	$$\frac{\partial \mathcal{L}_{BCE}}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\frac{\partial \mathcal{L}_{BCE}}{\partial \hat{y}_i}\frac{\partial \hat{y}_i}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)}\frac{\partial \hat{y}_i}{\partial W}$$

Categorical Cross-Entropy	$\frac{\partial \mathcal{L}_{CCE}}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}\frac{\partial \mathcal{L}_{CCE}}{\partial \hat{y}_{ij}}\frac{\partial \hat{y}_{ij}}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}\frac{y_{ij}}{\hat{y}_{ij}}\frac{\partial \hat{y}_{ij}}{\partial W}$

- Model memiliki implementasi metode **regularisasi L1** dan **L2**.
- Model memiliki implementasi **weight update** dengan menggunakan **gradient descent** untuk memperbarui bobot berdasarkan gradien yang telah dihitung, berikut persamaannya:

$$W_{new} = W_{old} - \alpha\left(\frac{\partial \mathcal{L}}{\partial W_{old}}\right)$$

$$\alpha = \text{Learning rate}$$

- Implementasi untuk pelatihan model harus memenuhi ketentuan berikut:
  - Dapat menerima parameter berikut:
    - Batch Size
    - Learning Rate
    - Jumlah Epoch
    - Verbose
- Verbose 0 berarti tidak menampilkan apa-apa selama pelatihan.
- Verbose 1 berarti hanya menampilkan progress bar beserta dengan kondisi training loss dan validation loss saat itu.
  - Proses pelatihan mengembalikan **histori dari proses pelatihan** yang berisi **training loss** dan **validation loss tiap epoch**

Pengujian

Gunakan dataset berikut untuk menguji model: global_student_placement_and_salary.

Pengujian dilakukan di **file .ipynb terpisah**.

1. **Hyperparameter**

   Analisis pengaruh beberapa **hyperparameter** sebagai berikut:

   - Pengaruh **depth (banyak layer)** dan **width (banyak neuron per layer)**
     - Pilih 3 variasi kombinasi width (depth tetap) dan 3 variasi depth (width semua layer tetap).
     - Bandingkan hasil akhir prediksinya.
     - Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan.
   - Pengaruh **fungsi aktivasi** hidden layer.
     - Tentukan base arsitektur (minimal 3 layer), pilih 1 layer sebagai layer pengetesan.

        - Lakukan eksperimen dengan menjalankan variasi base arsitektur dengan setiap fungsi aktivasi yang diimplementasikan **kecuali softmax** pada layer pilihan.
        - Bandingkan hasil akhir prediksinya.
        - Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan.
        - Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model.
    - Pengaruh **learning rate**.
        - Lakukan 3 variasi learning rate (nilainya dibebaskan).
        - Bandingkan hasil akhir prediksinya.
        - Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan.
        - Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model.

2. **Pengaruh Regularisasi**.
    - Lakukan eksperimen masing-masing 1 kali untuk model tanpa regularisasi, dengan regularisasi L1, dan dengan regularisasi L2.
    - Bandingkan hasil akhir prediksinya.
    - Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan.
    - Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model.
3. **Uji Perbandingan**.

    Analisis perbandingan hasil prediksi dengan library sklearn MLP.

        - Lakukan satu kali pelatihan dengan hyperparameter yang sama untuk kedua model.
        - Hyperparameter yang digunakan dibebaskan.
        - Bandingkan hasil akhir prediksinya saja.

Informasi Dataset

Pada Tugas Besar ini, kami diminta untuk melakukan **analisis dan pemodelan** menggunakan dataset *Global Student Placement & Salary Dataset* menggunakan **FFNN** yang sudah kalian buat. Dataset ini berisi kumpulan data global mengenai performa akademik mahasiswa serta outcome penempatan kerja dan gaji setelah lulus. Dataset dapat diakses **di sini**. Berikut deskripsi dari tiap fitur yang ada pada dataset yang akan digunakan (**placement_status** adalah kolom yang perlu diprediksi):

Column	Description	Type
cgpa	Cumulative Grade Point Average (skala 4.0–10.0)	Float
backlogs	Jumlah mata kuliah yang belum lulus	Integer
college_tier	Tingkatan institusi	Categorical
country	Negara kelulusan	Categorical

university_ranking_band	Kategori peringkat global universitas	Categorical
internship_count	Jumlah internship yang telah diselesaikan	Integer
aptitude_score	Skor tes kemampuan kuantitatif	Integer
communication_score	Skor kemampuan komunikasi	Integer
specialization	Bidang akademik	Categorical
industry	Industri tempat bekerja	Categorical
internship_quality_score	Skor kualitas pengalaman internship	Float
placement_status	Status penempatan kerja	Categorical

BAB II
PEMBAHASAN

Implementasi inti program *Forward Feed Neural Network* menggunakan **Python** dengan paradigma berorientasi objek. Terdapat 17 kelas tersebar di 5 file yang diimplementasi.

Pustaka yang digunakan adalah: *numpy*, *matplotlib*, *seaborn*, *pandas*, *scikit-learn*, dan *ipykernel*. Pustaka *scikit-learn* hanya digunakan untuk uji perbandingan model dan tidak digunakan dalam inti program.

Untuk kejelasan, bab ini hanya akan membahas deskripsi kelas pada tingkat tinggi dengan atribut dan metodenya, beserta penjelasan algoritma dan potongan kode sumber jika perlu. **Keseluruhan kode sumber akan dilampirkan pada lampiran**.

2.1. Kelas

2.1.1. Tensor

Tensor adalah struktur data array multidimensi yang menggeneralisasi skalar (0-D), vektor (1-D), dan matriks (2-D) ke tingkat dimensi yang lebih tinggi (*N*-D). Dalam *machine learning*, tensor digunakan sebagai input dan output dari suatu model *neural network*.

Di implemensi ini, Tensor memiliki fitur Auto Differentiation / AutoDiff, yaitu algoritma untuk melakukan diferensial secara otomatis menggunakan *chain rule*. Secara teknis, AutoDiff ini bekerja dengan membuat graf komputasi saat proses *forward propagation* atau *forward pass*, kemudian melakukan *chain rule* secara berulang saat *backward propagation*.

2.1.1.1. Atribut

data

Atribut *data* menyimpan isi dari Tensor. Tipe datanya berupa *array numpy*.

grad

Atribut *grad* menyimpan gradien dari isi Tensor. Tipe datanya berupa *array numpy*.

_prev

Atribut ini menyimpan *parent* dari Tensor itu sendiri. Berguna untuk membuat graf komputasi untuk AutoDiff saat *backward propagation*.

_backward

Atribut ini adalah sebuah fungsi yang menghitung *local gradient* dari Tensor.

2.1.1.2. Metode

__init__(self, data, _children, _op)

Metode untuk menginisialisasi sebuah Tensor.

__add__(self, other)
Metode *dunder* untuk menghitung pertambahan Tensor. Menggantikan sifat operasi '+' Tensor.

__matmul__(self, other)
Metode *dunder* untuk menghitung perkalian matriks Tensor. Menggantikan sifat operasi '@' Tensor.

__mul__(self, other)
Metode *dunder* untuk menghitung perkalian Tensor. Menggantikan sifat operasi '*' Tensor.

__neg__(self)
Metode *dunder* untuk menghitung negasi Tensor. Menggantikan sifat operasi '-' Tensor.

__sub__(self, other)
Metode *dunder* untuk menghitung pengurangan Tensor. Menggantikan sifat operasi '-' Tensor dengan objek lain.

__pow__(self, n)
Metode *dunder* untuk menghitung perpangkatan Tensor. Menggantikan sifat operasi '**' Tensor.

sum(self)
Metode untuk menghitung total penjumlahan data Tensor. Digunakan saat menghitung nilai regularisasi L1.

abs(self)
Metode untuk menghitung nilai absolut data Tensor. Digunakan saat menghitung nilai regularisasi L2.

backward(self)
Metode untuk memulai proses *backward propagation*. Proses *backward propagation* bekerja dengan menjelajah graf Tensor via atribut _prev, lalu membuat larik yang terurut secara topologi untuk memastikan proses diferensiasi dengan *chain rule* berjalan dengan benar.

```
    def backward(self):
        """ Memulai proses backpropagation dari node ini. """
        self.grad = np.ones_like(self.data)

        topo = []
        visited = set()

        def build_topo(v):
            if v not in visited:
                visited.add(v)
                for child in v._prev:
                    build_topo(child)
                topo.append(v)
```

```
        build_topo(self)

        for node in reversed(topo):
            node._backward()
```

Terlihat di potongan kode sumber, fungsi build_topo beserta variabel topo dan visited didefinisikan untuk membantu proses traversal. Setelah variabel topo terisi dengan benar, topo dibalik. Kemudian untuk tiap simpul di larik tersebut akan dipanggil fungsi _backward, yang akan melakukan proses diferensial.

2.1.2. Module

Module adalah objek dasar dalam sistem *neural network*, Objek dasar ini mampu menyimpan barisan parameter atau nilai (atau Tensor) dengan gradiennya, dan mampu me-*reset* gradien barisan parameter tersebut menjadi 0.

2.1.2.1. Atribut

grad

Atribut *grad* menyimpan gradien dari Module. Tipe datanya bisa berupa *array numpy* atau Tensor.

2.1.2.2. Metode

zero_grad(self)

Metode untuk me-*reset* nilai gradien (*grad*) menjadi 0.

parameters(self)

Fungsi untuk mendapatkan barisan parameter yang disimpan oleh objek Module.

2.1.3. Layer

Layer adalah representasi *layer* atau lapisan dari sebuah *neural network*. Kelas ini merupakan turunan dari kelas Module.

2.1.3.1. Atribut

activation

Atribut untuk menyimpan objek BaseActivation (dan turunannya). Merepresentasikan fungsi aktivasi dari *layer* ini.

activation_name

Nama dari objek BaseActivation yang disimpan di atribut *activation*.

b

Atribut untuk menyimpan bobot bias di *layer* ini. Tipe datanya berupa Tensor.

w

Atribut untuk menyimpan bobot simpul selain bias di *layer* ini. Tipe datanya berupa Tensor.

random_state

Atribut untuk menyimpan *seed* yang digunakan sebagai *seed* modul randomisasi milik *numpy*.

2.1.3.2. Metode

__init__(self, nin, nout, activation_name, init_method, random_state)

Metode inisialisasi *layer* dalam *neural network*. Menerima parameter berupa ukuran data yang masuk dan keluar *layer*, juga init_method untuk menentukan bobot simpul (selain bias) awal.

Metode inisialisasi bobot yang diimplementasi adalah:

- Zero: Mengatur semua bobot menjadi angka nol
- Uniform: Mengambil nilai bobot secara acak dengan peluang yang sama di dalam rentang angka yang ditentukan
- Normal: Menentukan nilai bobot berdasarkan distribusi normal dengan rata-rata dan varians tertentu agar nilai awal tersebar secara alami di sekitar titik pusat
- Xavier: Menetapkan standar deviasi bobot sebesar akar kuadrat dari satu dibagi jumlah unit masukan
- He: Menetapkan standar deviasi bobot sebesar akar kuadrat dari dua dibagi jumlah unit masukan

parameters(self)

Metode *getter* untuk mengembalikan larik yang berisi bobot bias dan bobot simpul lainnya.

__call__(self, x)

Metode *dunder* untuk menggantikan sifat operasi pemanggilan '()' objek Layer.

```
def __call__(self, x):
    return self.activation(x @ self.w + self.b)
```

Operasi pemanggilan *layer* merepresentasikan proses pengoperasian fungsi aktivasi pada prediksi bobot yang ada di *layer* ini, dimana prediksi dari *layer* $n - 1$ menuju *layer* n didefinisikan sebagai:

$$o_n\left(\overline{x}_{n-1}\right) = f_j\left(\overline{x}_{n-1} \cdot \overline{w}_n + \overline{b}_n\right)$$

Dimana:

o_n	:	Output dari *layer* n
$\overline{x}_{n-1}$	:	Input ke *layer* n, yang berasal dari *layer* $n - 1$.
f_n	:	Fungsi aktivasi *layer* n

$\overline{w}_n$: Bobot menuju *layer* n

$\overline{b}_n$: Bobot bias menuju *layer* n

2.1.4. FFNN

FFNN adalah kelas representasi model Forward Feed Neural Network. Kelas ini adalah turunan dari Module dan berisi kumpulan Layer.

2.1.4.1. Atribut

init_method

Atribut untuk menyimpan metode inisialisasi bobot Layer. Disimpan untuk keperluan *load* dan *save*.

layers

Atribut larik yang menyimpan kumpulan Layer.

layers_config

Atribut larik yang menyimpan konfigurasi ukuran Layer. Disimpan untuk keperluan *load* dan *save*.

loss_name

Atribut yang menyimpan nama *loss function* yang digunakan. Disimpan untuk keperluan *load* dan *save*.

loss_fn

Atribut yang menyimpan objek fungsional *loss function* sesuai *loss_name*

random_state

Atribut untuk menyimpan *seed* yang digunakan sebagai *seed* modul randomisasi milik *numpy*.

2.1.4.2. Metode

__init__(self, layers_config, activations, loss_function, init_method, random_state)

Metode untuk inisialisasi model FFNN. Parameter yang diperlukan berupa konfigurasi *layer*, fungsi-fungsi aktivasi, *loss function*, metode inisialisasi, dan *seed* untuk randomisasi.

parameters(self)

Metode *getter* untuk mendapatkan keseluruhan parameter atau nilai bobot di model FFNN.

fit(self, X_train, y_train, X_val, y_val, solver, batch_size, lr, l1_lambda, l2_lambda, epochs, verbose)

Metode untuk memulai proses pelatihan FFNN. Menerima parameter data input output latihan dan validasi, metode solver/optimisasi, ukuran *batch*,

learning rate, nilai regularisasi L1 dan L2, jumlah *epoch*, serta *verbose flag* untuk keperluan *logging*.

Algoritma pelatihan FFNN secara garis besar dapat dijabarkan seperti ini:

1. Untuk tiap iterasi epoch:
    a. Acak data input dan bagi ke ukuran *batch*.
    b. Untuk tiap *batch:*
        i. Lakukan proses *forward propagation.*
        ii. Hitung *loss* dengan *loss function* yang dipilih.
        iii. Tambahkan regularisasi *loss* L1 dan L2 jika perlu.
        iv. Lakukan *backward propagation* dengan AutoDiff untuk mendapatkan nilai gradien.
        v. Perbarui bobot dengan fungsi step() milik optimizer.
    c. Simpan nilai *loss* rata-rata batch ke histori.
    d. Jika ingin mencari nilai *validation loss*:
        i. Lakukan *forward propagation* dengan set validasi.
        ii. Hitung *loss* dengan *loss function* yang dipilih.
        iii. Tambahkan regularisasi *loss* L1 dan L2 jika perlu.
        iv. Simpan nilai *loss* validasi ke histori.
2. Kembalikan histori.

Proses pelatihan akan mengubah bobot yang ada di *layer-layer* dan akan mengembalikan histori yang berisi kurva nilai *loss* pelatihan dan validasi (jika perlu).

predict(self, X)

Fungsi untuk mendapatkan output atau prediksi nilai dari input data X. Prosesnya sama dengan *forward propagation*. Mengembalikan barisan nilai output.

forward(self, x)

Fungsi untuk melakukan *forward propagation* dengan input data x. Mengembalikan Tensor hasil *forward propagation*.

save(self, path)

Metode untuk menyimpan *state* atau keadaan model FFNN saat ini ke file format "pickle" (.pkl).

load(self, path)

Metode untuk memuat s*tate* atau keadaan model FFNN dari file format "pickle" (.pkl) ke objek model di memori.

load_new(cls, path)

Metode untuk memuat s*tate* atau keadaan model FFNN dari file format "pickle" (.pkl) ke objek model FFNN yang baru.

plot_weights_distribution(self, layer_indices, bins)

Metode untuk menampilkan distribusi bobot dalam bentuk histogram. Jumlah bin histogram dijadikan parameter metode dengan nilai standar 30. Visualisasi menggunakan pustaka *matplotlib*.

plot_gradients_distribution(self, layer_indices, bins)

Metode untuk menampilkan distribusi gradien bobot dalam bentuk histogram. Jumlah bin histogram dijadikan parameter metode dengan nilai standar 30. Visualisasi menggunakan pustaka *matplotlib*.

2.1.5. BaseActivation

Kelas BaseActivation adalah kelas abstrak untuk semua kelas turunan yang merepresentasikan fungsi aktivasi.

2.1.5.1. Metode

forward(self, x)

Fungsi abstrak (tidak ada implementasi). Fungsi ini mengembalikan Tensor. Kelas turunan harus mengimplementasi fungsi ini yang akan memberlakukan fungsi aktivasi ke input data x.

Selain mengimplementasi fungsi aktivasi, kelas turunan juga harus mengimplementasikan fungsi turunan dari fungsi aktivasi sebagai atribut fungsi _backward dari Tensor yang dikembalikan.

__call__(self, x)

Fungsi *dunder* sebagai *wrapper* untuk fungsi *forward*, agar bisa dipanggil langsung dengan memanggil pada objek/instansi kelas.

2.1.6. Linear

Kelas fungsi aktivasi linier. Diturunkan dari kelas BaseActivation.

2.1.6.1. Metode

forward(self, x)

Fungsi untuk menjalankan fungsi aktivasi linier pada data x. Fungsi aktivasi linier dijabarkan sebagai berikut:

$$Linear(x) = x$$

Karena nilai input x tidak diubah, maka fungsi aktivasi ini berisiko menyebabkan *overflow*. Tidak disarankan digunakan untuk aktivasi *output layer*.

Turunan fungsi aktivasi linier adalah:

$$\frac{d\big(Linear(x)\big)}{dx} = 1$$

karena fungsi linier memiliki gradien yang konstan.

2.1.7. ReLU

Kelas fungsi aktivasi ReLU (*Rectified Linear Unit*). Diturunkan dari kelas BaseActivation.

2.1.7.1. Metode

forward(self, x)

Fungsi untuk menjalankan fungsi aktivasi ReLU pada data x. Fungsi aktivasi ReLU dijabarkan sebagai berikut:

$$ReLU(x) = \max(0, x)$$

Nilai yang dihasilkan adalah x jika lebih dari 0, dan 0 jika kurang dari sama dengan 0. Fungsi aktivasi ini populer digunakan karena dapat mengurangi masalah gradien yang menjadi sangat kecil selama pelatihan (*vanishing gradient*) sehingga menyulitkan proses pembelajaran.

Secara matematis, turunan fungsi aktivasi ReLU adalah:

$$\frac{d(ReLU(x))}{dx} = \begin{cases} 0, & for\ x < 0 \\ 1, & for\ x > 0 \end{cases},$$

dimana turunannya tidak terdefinisi untuk $x = 0$. Namun, konvensi teknis menyatakan bahwa turunannya adalah 1 untuk $x > 0$, sehingga selain itu nilai turunannya adalah 0. Sehingga untuk implementasi, turunannya adalah:

$$\frac{d\big(ReLU(x)\big)}{dx} = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$$

2.1.8. Sigmoid

Kelas fungsi aktivasi sigmoid. Diturunkan dari kelas BaseActivation.

2.1.8.1. Metode

forward(self, x)

Fungsi untuk menjalankan fungsi aktivasi sigmoid pada data x. Fungsi aktivasi sigmoid dijabarkan sebagai berikut:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Domain dari fungsi aktivasi ini adalah $(-\infty, \infty)$, denga *range* $(0, 1)$. Oleh karena itu, biasanya fungsi ini cocok digunakan sebagai fungsi aktivasi di *output layer*.

Turunan fungsi aktivasi sigmoid adalah:

$$\frac{d(\sigma(x))}{dx} = \sigma(x)(1 - \sigma(x))$$

2.1.9. TanH

Kelas fungsi aktivasi tanh (*hyperbolic tangent*). Diturunkan dari kelas BaseActivation.

2.1.9.1. Metode

forward(self, x)

Fungsi untuk menjalankan fungsi aktivasi tanh pada data x. Fungsi aktivasi tanh dijabarkan sebagai berikut:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Domain dari fungsi aktivasi tanh adalah $(-\infty, \infty)$, denga *range* $(-1, 1)$. Oleh karena itu, biasanya fungsi ini digunakan di *hidden layer* pada *neural network* yang tidak kompleks.

Turunan fungsi aktivasi tanh adalah:

$$\frac{d(tanh(x))}{dx} = 1 - tanh^2(x)$$

2.1.10. Softmax

Kelas fungsi aktivasi Softmax. Diturunkan dari kelas BaseActivation.

2.1.10.1. Metode

forward(self, x)

Fungsi untuk menjalankan fungsi aktivasi softmax pada data x. Fungsi aktivasi softmax dijabarkan sebagai berikut:

$$\text{Untuk vector } \vec{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n,$$
$$softmax(\vec{x})_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

Fungsi ini mengkonversi vektor dari *neural network* menjadi distribusi probabilitas, dimana *range* nilainya adalah $(0, 1)$, dan totalnya adalah 1. Biasanya digunakan di *output layer* dari neural network untuk multi-klasifikasi.

Turunan fungsi aktivasi softmax dijabarkan sebagai berikut:

Untuk vector $\vec{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$,

$$\frac{d(softmax(\vec{x})_i)}{d\vec{x}} = \begin{bmatrix} \frac{\partial(softmax(\vec{x})_1)}{\partial x_1} & \cdots & \frac{\partial(softmax(\vec{x})_1)}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial(softmax(\vec{x})_n)}{\partial x_1} & \cdots & \frac{\partial(softmax(\vec{x})_n)}{\partial x_n} \end{bmatrix}$$

Dimana untuk $i, j \in \{1, \dots, n\}$,

$$\frac{\partial(softmax(\vec{x})_i)}{\partial x_j} = softmax(\vec{x})_i(\delta_{i,j} - softmax(\vec{x})_j)$$

$$\delta_{i,j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

2.1.11. LeakyReLU

Kelas fungsi aktivasi Leaky ReLU. Diturunkan dari kelas BaseActivation

2.1.11.1. Atribut

a

Nilai konstanta dalam *range* $(0, 1)$, berfungsi sebagai pengali nilai x ketika $x < 0$.

2.1.11.2. Metode

__init__(self, a)

Metode untuk inisialisasi objek Leaky ReLU. Menerima a sebagai parameter, dengan nilai standar = 0.01.

forward(self, x)

Fungsi untuk menjalankan fungsi aktivasi Leaky ReLU pada data x. Fungsi aktivasi Leaky ReLU dijabarkan sebagai berikut:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases},$$

Berbeda dengan ReLU, fungsi aktivasi ini mencegah suatu *neuron* mati dengan memboleh gradien kecil untuk $x < 0$. Hal ini dapat membuat pembelajaran lebih efisien dan stabil untuk kasus dimana ReLU gagal, seperti *neural network* yang dalam.

Turunan fungsi aktivasi Leaky ReLU adalah:

$$\phi'(x) = \begin{cases} \alpha & x \leq 0 \\ 1 & x > 0 \end{cases}$$

2.1.12. BaseLoss

Kelas BaseLoss adalah kelas abstrak untuk semua kelas turunan yang merepresentasikan *loss function*.

2.1.12.1. Metode

forward(self, x)

Fungsi abstrak (tidak ada implementasi). Fungsi ini mengembalikan Tensor. Kelas turunan harus mengimplementasi fungsi ini yang akan memberlakukan *loss function* ke input data x.

Selain mengimplementasi *loss function*, kelas turunan juga harus mengimplementasikan fungsi turunan dari *loss function* sebagai atribut fungsi _backward dari Tensor yang dikembalikan.

__call__(self, x)

Fungsi *dunder* sebagai *wrapper* untuk fungsi *forward*, agar bisa dipanggil langsung dengan memanggil pada objek/instansi kelas.

2.1.13. MSELoss

Kelas *loss function* Mean Square Error (MSE). Diturunkan dari kelas BaseLoss.

2.1.13.1. Metode

forward(self, x)

Fungsi untuk menjalankan fungsi MSE pada data x. Fungsi MSE dijabarkan sebagai berikut:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Turunan fungsi MSE terhadap bobot adalah:

$$\frac{\partial \mathcal{L}_{MSE}}{\partial W} = -\frac{2}{n}\sum_{i=1}^{n}\frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}_i}\frac{\partial \hat{y}_i}{\partial W} = -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)\frac{\partial \hat{y}_i}{\partial W}$$

2.1.14. BinaryClassEntropy

Kelas *loss function* Binary Class Entropy (BCE). Diturunkan dari kelas BaseLoss.

2.1.14.1. Metode

forward(self, x)

Fungsi untuk menjalankan fungsi BCE pada data x. Fungsi BCE dijabarkan sebagai berikut:

$$\mathcal{L}_{BCE} = -\frac{1}{n}\sum_{i=1}^{n}(y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i))$$

y_i = Actual binary label (0 or 1)

$\hat{y}_i$ = Predicted value of y_i

n = Batch size

Turunan fungsi BCE terhadap bobot adalah:

$$\frac{\partial \mathcal{L}_{BCE}}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\frac{\partial \mathcal{L}_{BCE}}{\partial \hat{y}_i}\frac{\partial \hat{y}_i}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)}\frac{\partial \hat{y}_i}{\partial W}$$

2.1.15. CategoricalClassEntropy

Kelas *loss function* Categorical Class Entropy (CCE). Diturunkan dari kelas BaseLoss.

2.1.15.1. Metode

forward(self, x)

Fungsi untuk menjalankan fungsi CCE pada data x. Fungsi CCE dijabarkan sebagai berikut:

$$\mathcal{L}_{CCE} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}(y_{ij} \log \hat{y}_{ij})$$

y_{ij} = Actual value of instance i for class j

$\hat{y}_{ij}$ = Predicted value of y_{ij}

C = Number of classes

n = Batch size

Turunan fungsi BCE terhadap bobot adalah:

$$\frac{\partial \mathcal{L}_{CCE}}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}\frac{\partial \mathcal{L}_{CCE}}{\partial \hat{y}_{ij}}\frac{\partial \hat{y}_{ij}}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}\frac{y_{ij}}{\hat{y}_{ij}}\frac{\partial \hat{y}_{ij}}{\partial W}$$

2.1.16. BaseOptimizer

Kelas BaseOptimizer adalah kelas abstrak untuk semua kelas turunan yang merepresentasikan optimizer, yaitu objek yang memperbarui bobot / parameter *neural network* di tiap *epoch*.

2.1.16.1. Metode

step(self)

Fungsi abstrak. Kelas turunan harus mengimplementasi fungsi ini, yang akan memperbarui bobot / parameter *neural network*.

2.1.17. SGD

Kelas optimasi Stochastic Gradient Descent (SGD), Kelas ini diturunkan dari BaseOptimizer.

2.1.17.1. Atribut

parameters

Atribut yang menyimpan seluruh bobot / parameter suatu *neural network*.

lr

Atribut *learning rate*. Nilai ini sebaiknya dalam *range* $(0, 1)$.

2.1.17.2. Metode

step(self)

Fungsi step akan memperbarui bobot / parameter *neural network* dengan algoritma SGD. Algoritma SGD dijabarkan sebagai berikut:

1. Isi parameter / bobot model awal w dan *learning rate* η
2. Acak data input dan bagi ke ukuran *batch*.
3. Untuk tiap *batch:*
    a. Lakukan proses *forward propagation*
    b. Lakukan proses *backward propagation* sehingga nilai gradien didapat.
    c. Perbarui bobot dengan rumus SGD.

Rumus pembaruan bobot dijabarkan sebagai berikut:

$$w_t = w_{t-1} - \eta \nabla Q\left(w_{t-1}\right) \quad ,$$

dimana $\nabla Q\left(w_{t-1}\right)$ adalah gradien *loss function* terhadap bobot sebelum pembaruan bobot.

Terlihat dari *pseudocode* sebelumnya, algoritma SGD beririsan dengan bagaimana proses pelatihan FFNN bekerja, sehingga SGD cocok digunakan.

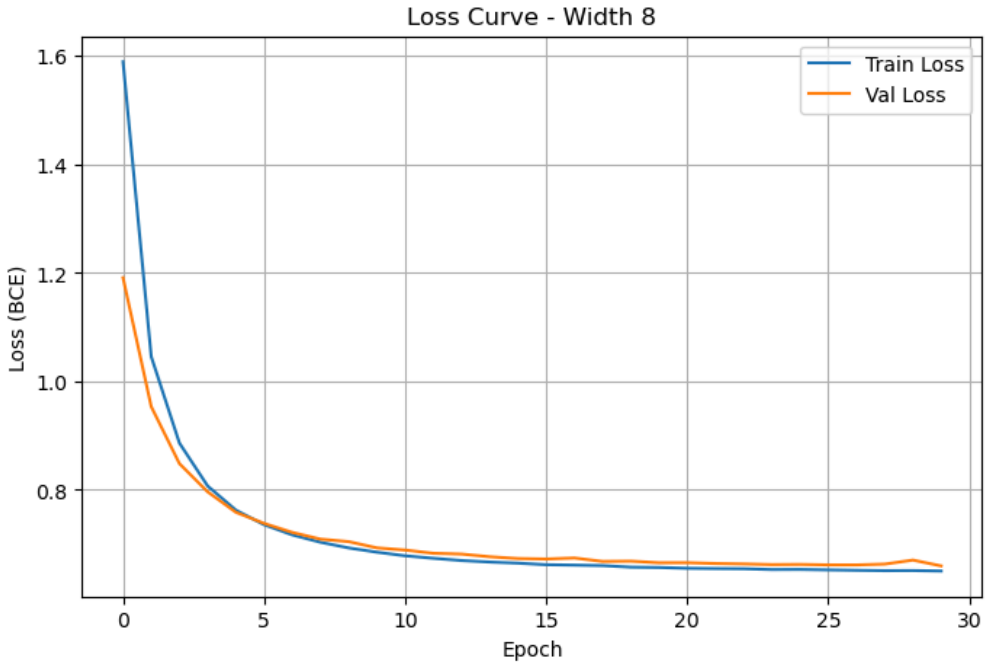
2.2. Hasil Eksperimen dan Analisis

Hasil

2.2.1. Pengaruh Depth dan Width Layer

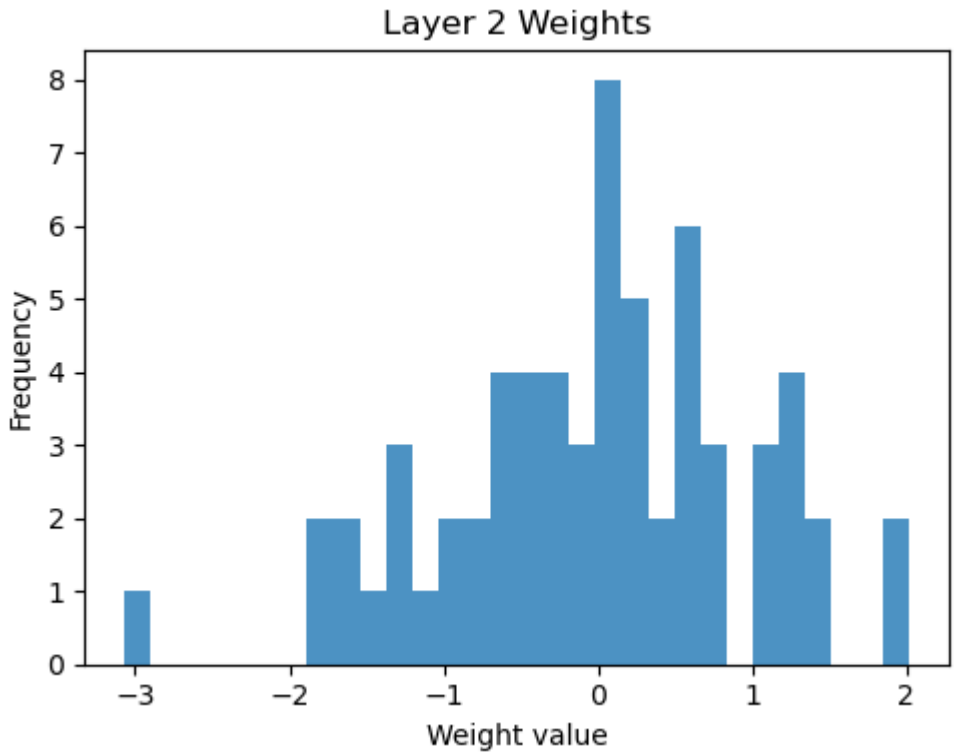
Metrik	Width 8	Width 16	Width 32
accuracy	0.61	0.67	0.67
Macro F1	0.47	0.64	0.67
Weighted F1	0.53	0.66	0.67
Placed (1) Precision	0.62	0.71	0.78
Placed (1) Recall	0.93	0.77	0.64
Not Placed (0) Precision	0.55	0.59	0.56
Not Placed (0) Recall	0.13	0.51	0.72

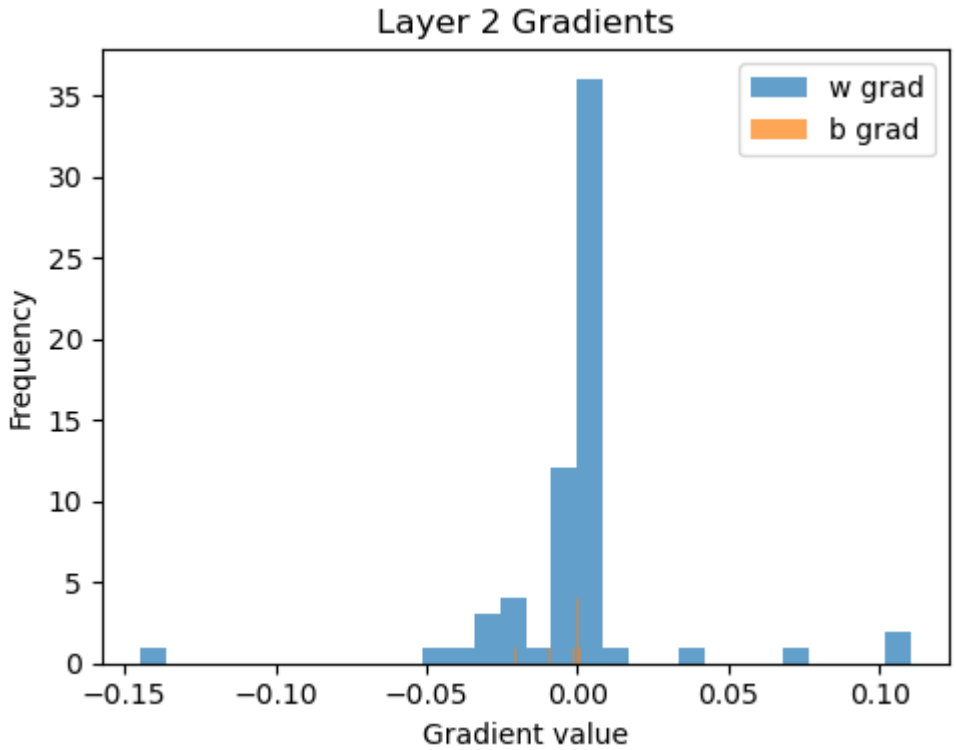
Pada eksperimen *Width*, peningkatan dari 8 ke 32 neuron memberikan kenaikan performa yang cukup signifikan, *accuracy* naik dari 0.61 ke 0.67 dan *Macro F1* melonjak dari 0.47 ke 0.67 yang merupakan juga peningkatan dari saat width 16, yaitu 0.64. Ini membuktikan bahwa dataset yang dipakai membutuhkan kapasitas representasi (jumlah neuron) yang lebih besar untuk mengklasifikasikan fitur secara efektif.



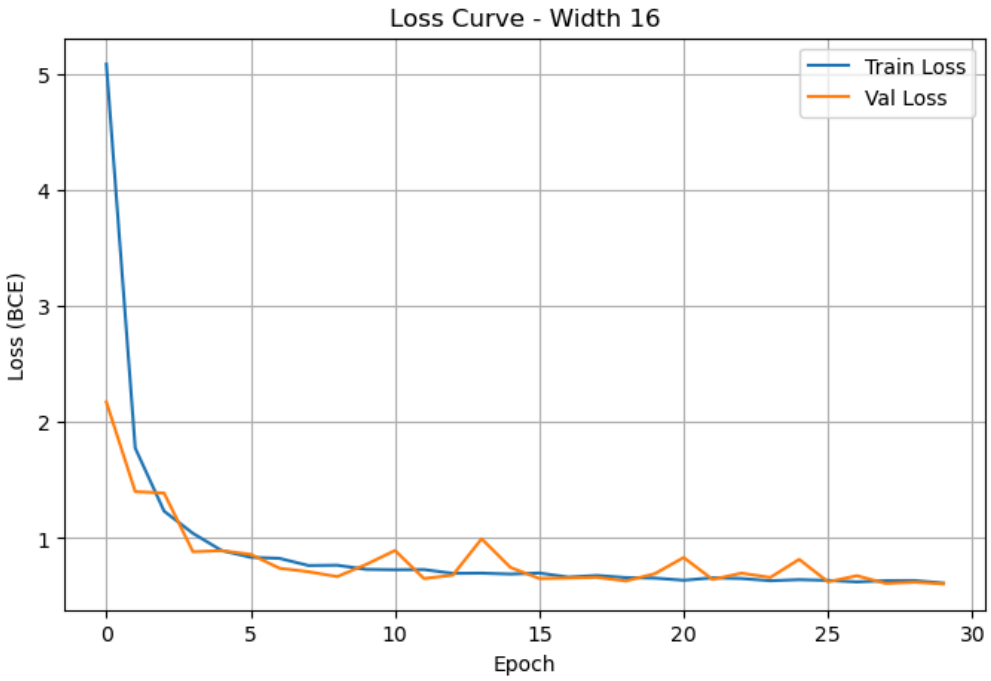
Kurva loss pada model dengan *Width* 8 menunjukkan penurunan awal yang cukup cepat namun dengan segera mencapai titik *plateau* pada nilai loss yang masih cukup tinggi (berhenti di 0.6505). Kurva train loss dan validation loss bergerak sangat berdekatan yang berarti tidak terjadi overfitting. Namun,

tingginya batas bawah nilai loss ini merupakan indikator kuat terjadinya underfitting, kapasitas arsitektur terlalu sempit untuk memahami kompleksitas data.

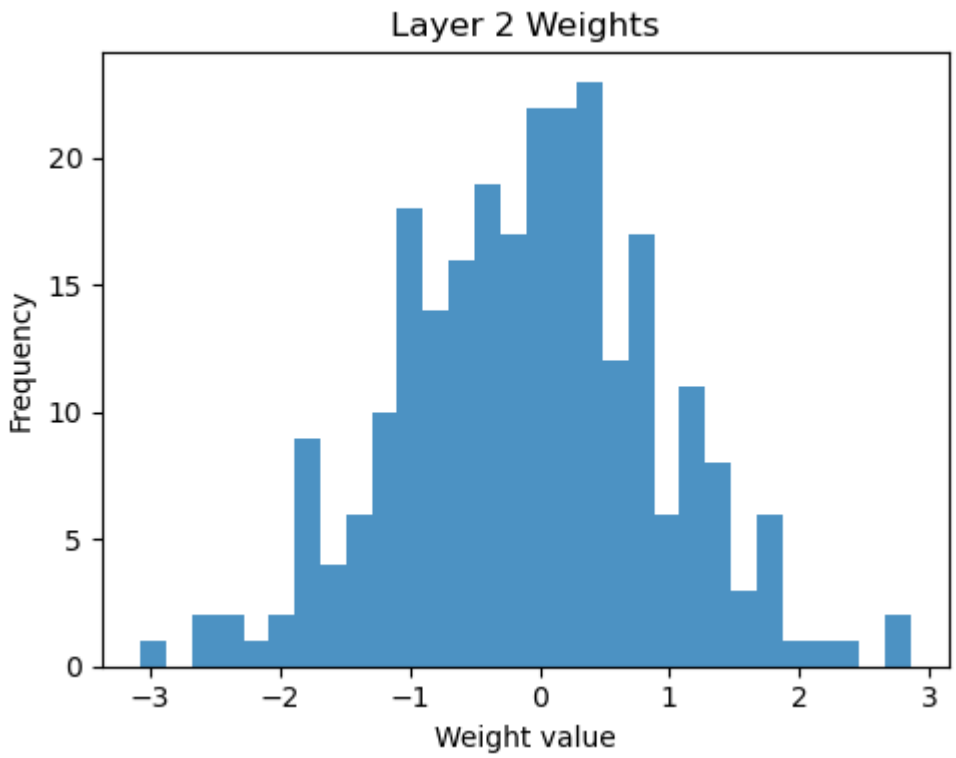


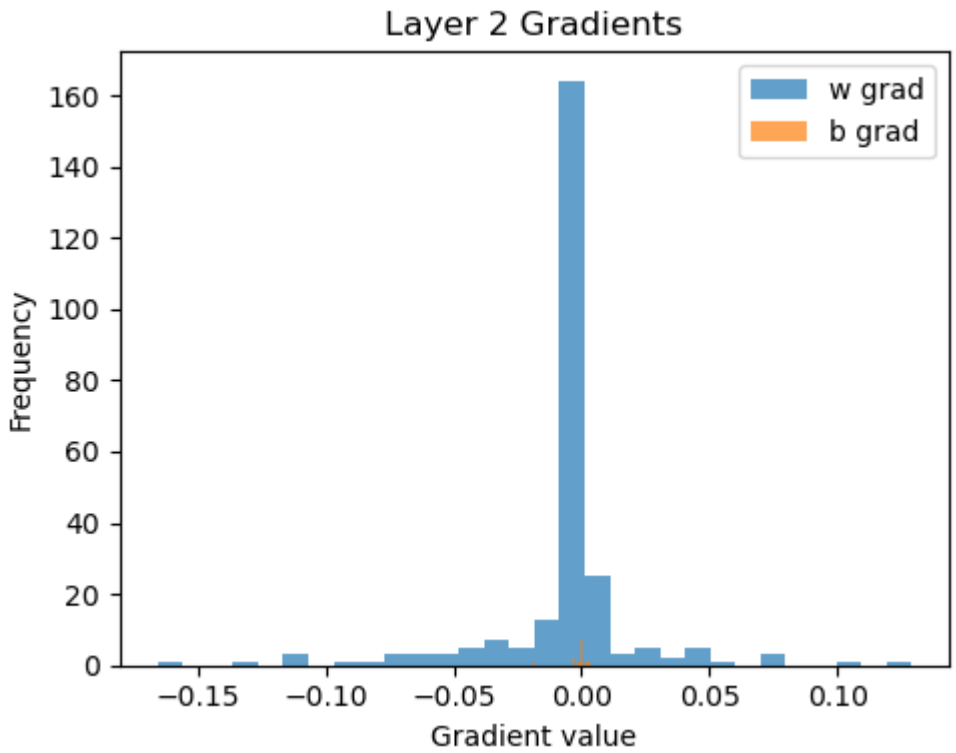


Distribusi bobot *Layer 2* pada *Width 8* memperlihatkan histogram yang sangat renggang, patah-patah, dan frekuensi kemunculan nilai yang sangat kecil. Distribusi gradiennya juga memperlihatkan rentang yang sempit. Ini adalah bukti bahwa sedikitnya jumlah neuron membuat beban komputasi tidak terbagi rata, sehingga neuron gagal mengekstraksi variasi fitur dari dataset.

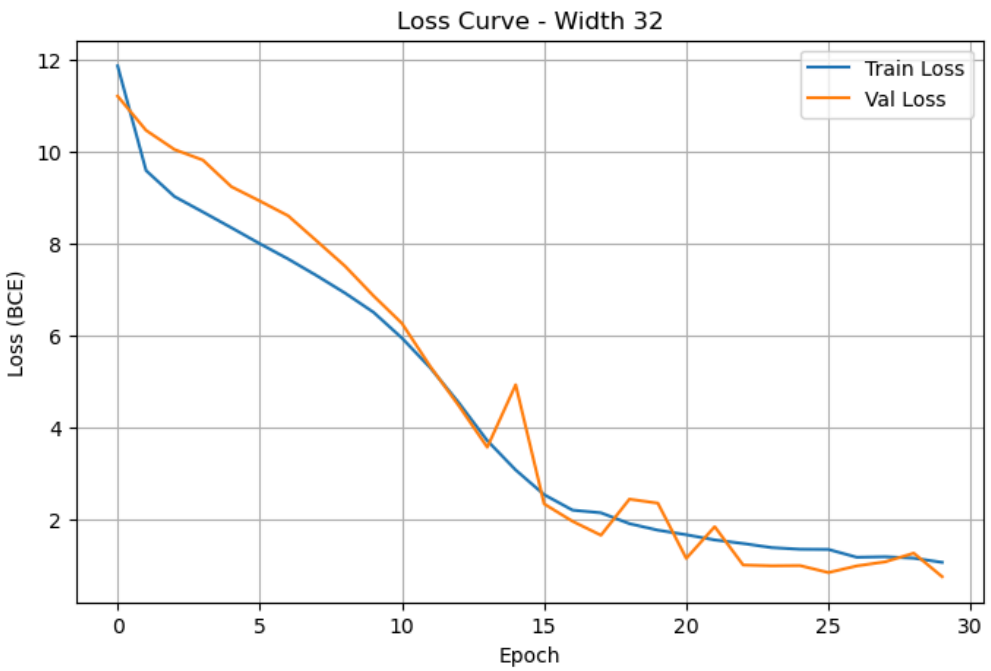


Pada Width 16, kurva loss menunjukkan penurunan yang lebih baik dan lebih dalam dibandingkan *Width 8*, mencapai ekuilibrium di nilai loss yang lebih rendah (0.6128). Terdapat sedikit pelebaran jarak (gap) antara train dan validation di pertengahan epoch, yang merupakan pertanda wajar bahwa model mulai memiliki kapasitas memori yang cukup untuk benar-benar mempelajari fitur spesifik dari training data.

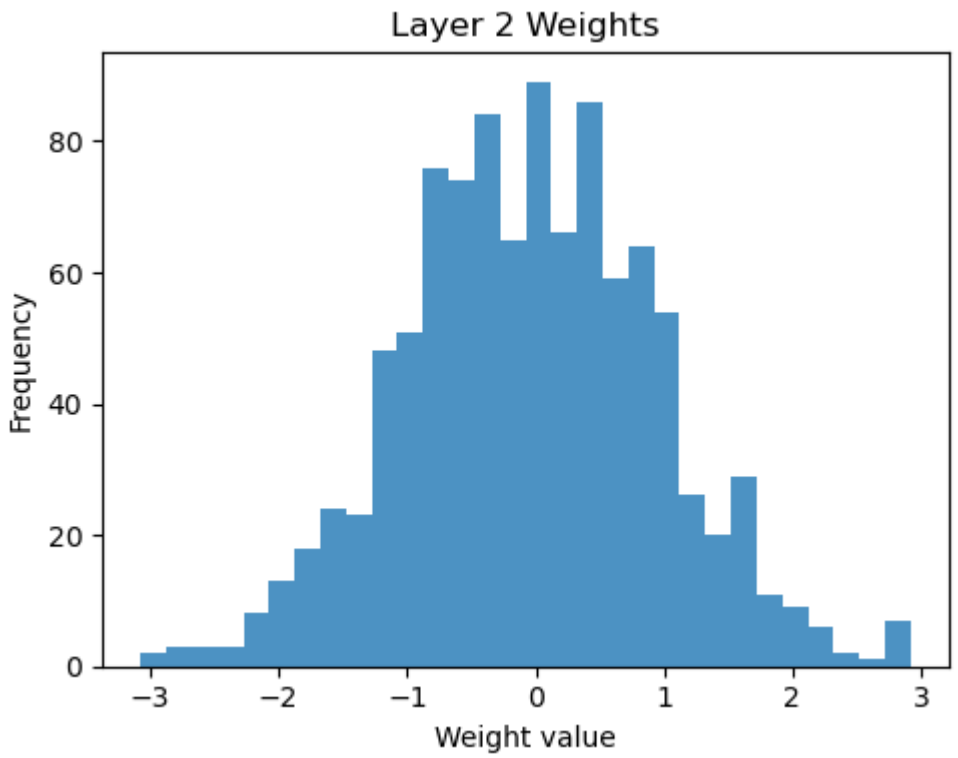


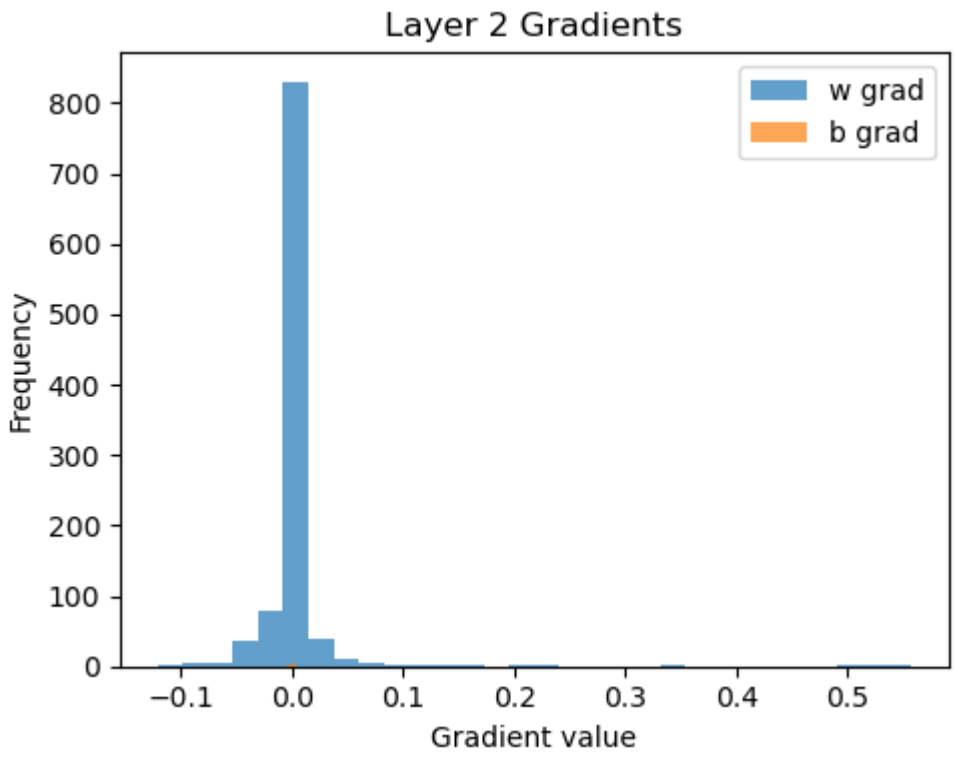


Histogram bobot pada *Layer 2* untuk *Width 16* mulai membentuk kurva *bell-shape* yang lebih teratur dengan peningkatan frekuensi hingga skala 20. Sebaran gradien yang lebih aktif menunjukkan bahwa penambahan 8 neuron ekstra ini sukses mendistribusikan aliran sinyal backpropagation dengan lebih baik ke seluruh node di dalam layer tersebut.



Kurva loss Width 32 mengalami penurunan yang sangat masif dan agresif (turun dari skala 12 hingga ke 1). Terdapat fluktuasi atau lonjakan yang cukup tajam pada validation loss di sekitar epoch 15 hingga 20. Fluktuasi ini adalah karakteristik dari model berkapasitas besar; model menjadi lebih "sensitif" terhadap noise dari mini-batch data, sebelum akhirnya konvergen dengan sangat baik di epoch akhir.

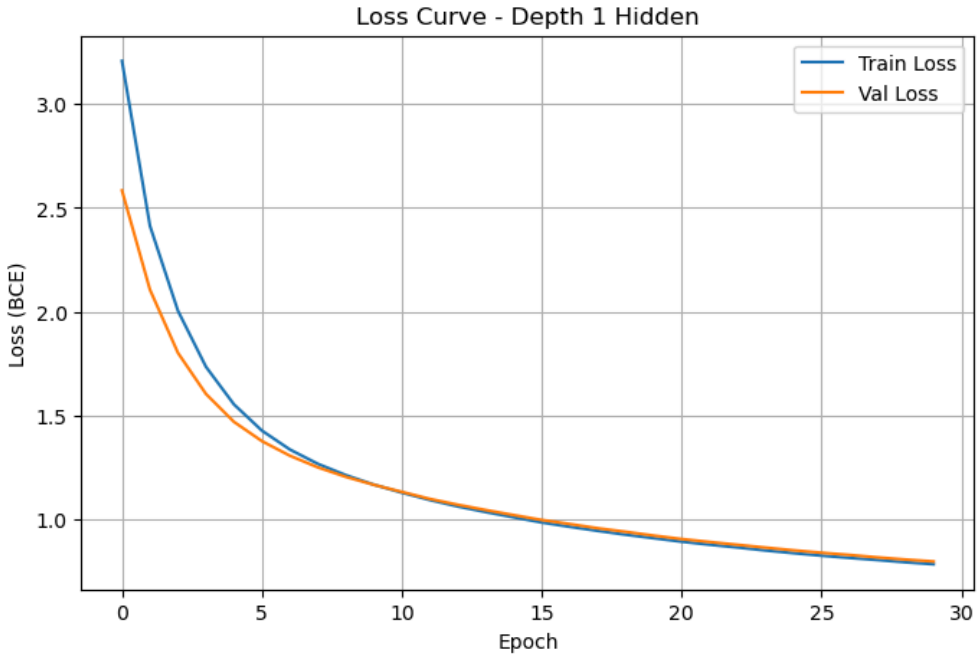




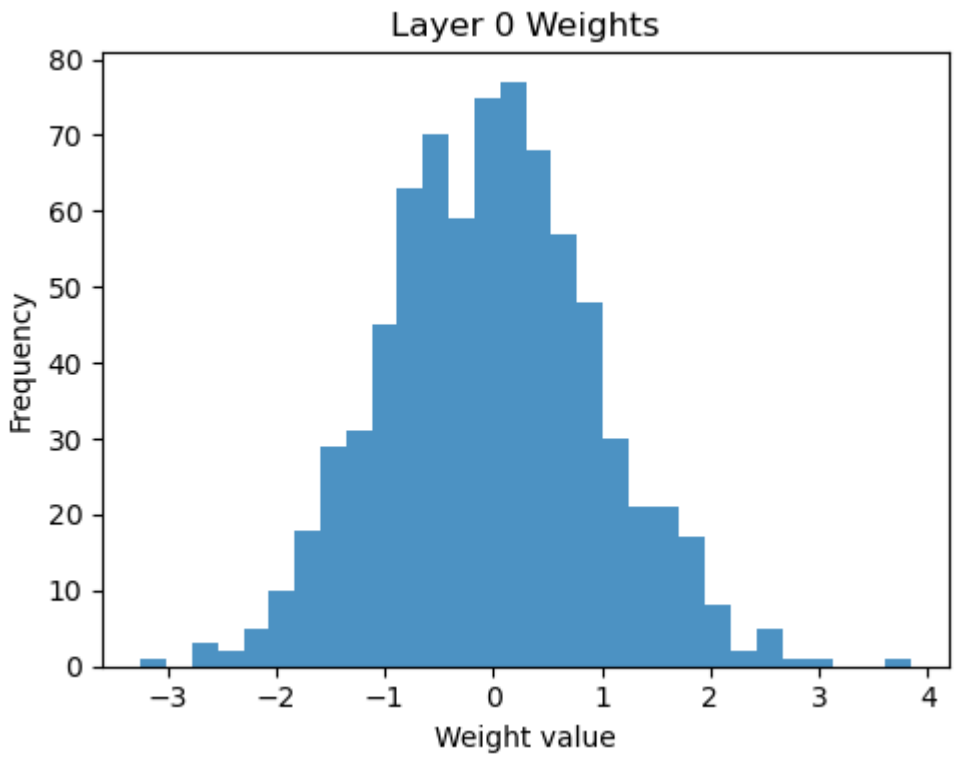
Distribusi bobot Layer 2 sangat padat dan solid dengan frekuensi mencapai angka 80, menandakan pemanfaatan kapasitas neuron yang maksimal. Distribusi gradien memuncak sangat tajam (frekuensi ratusan) tepat di angka nol. Pemusatan gradien ekstrem di nol pada *layer* akhir ini menandakan bahwa pada akhir pelatihan, model telah mencapai konvergensi yang matang di mana bobot tidak lagi memerlukan pembaruan yang besar.

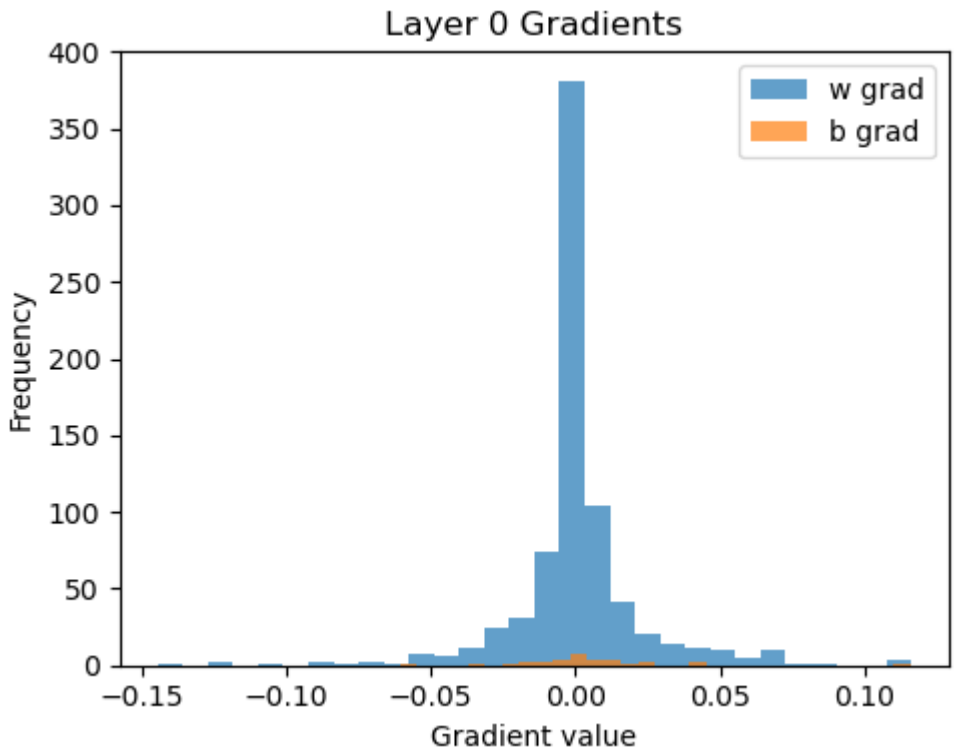
Metrik	*Depth* 1	*Depth* 2	*Depth* 3
accuracy	0.68	0.69	0.67
Macro F1	0.66	0.66	0.67
Weighted F1	0.68	0.68	0.67
Placed (1) Precision	0.72	0.71	0.78
Placed (1) Recall	0.76	0.81	0.64
Not Placed (0) Precision	0.60	0.63	0.56
Not Placed (0) Recall	0.56	0.51	0.72

Pada eksperimen variasi *Depth*, model dengan 2 hidden layer menghasilkan akurasi tertinggi sebesar 0.69, dibandingkan dengan 1 hidden layer (0.68) dan 3 hidden layer (0.67). Hasil ini menunjukkan bahwa peningkatan *Depth* meningkatkan kemampuan model dalam merepresentasikan hubungan non-linier hingga batas tertentu. Namun, penambahan layer yang berlebihan dapat menurunkan performa karena penurunan efektivitas propagasi gradien selama proses *backpropagation*, yang berkaitan dengan fenomena *vanishing gradient*.

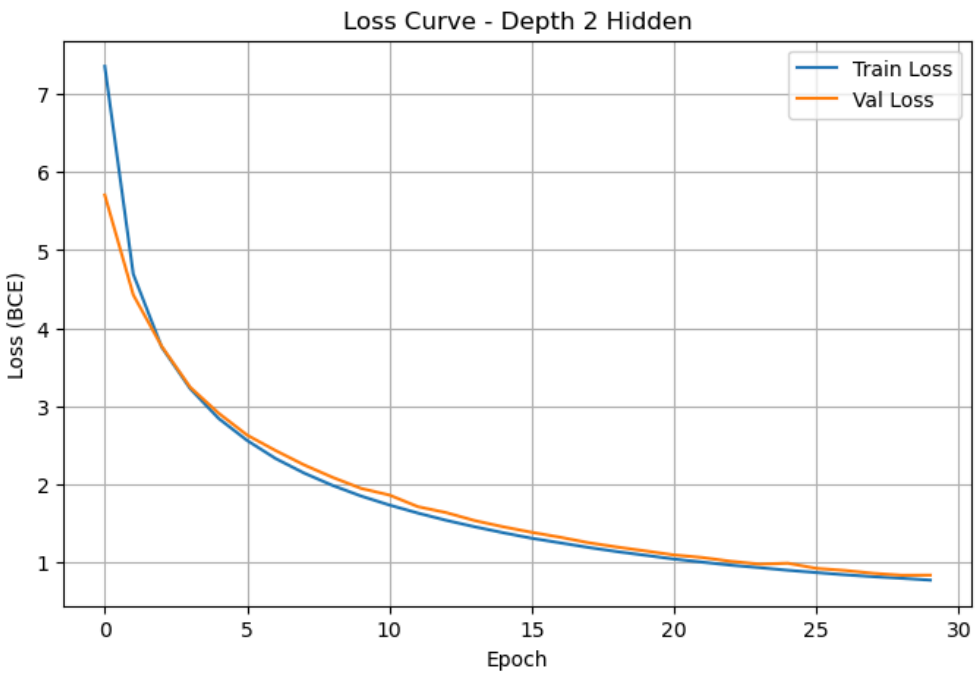


Kurva *loss* pada model dengan 1 hidden layer menampilkan peluruhan eksponensial yang sangat mulus dan stabil tanpa fluktuasi berarti pada data validasi. Kesempurnaan bentuk kurva ini menunjukkan proses optimasi yang sangat mudah (karena ruang parameter lebih sederhana), meskipun batas kemampuan learning-nya tertahan di ekuilibrium tertentu.

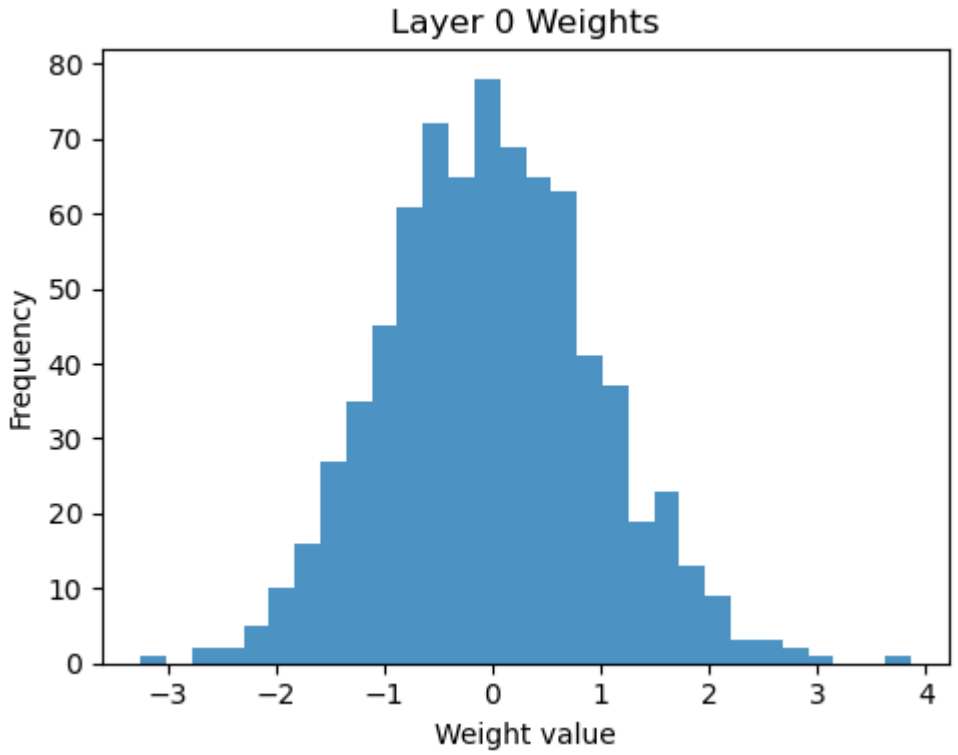


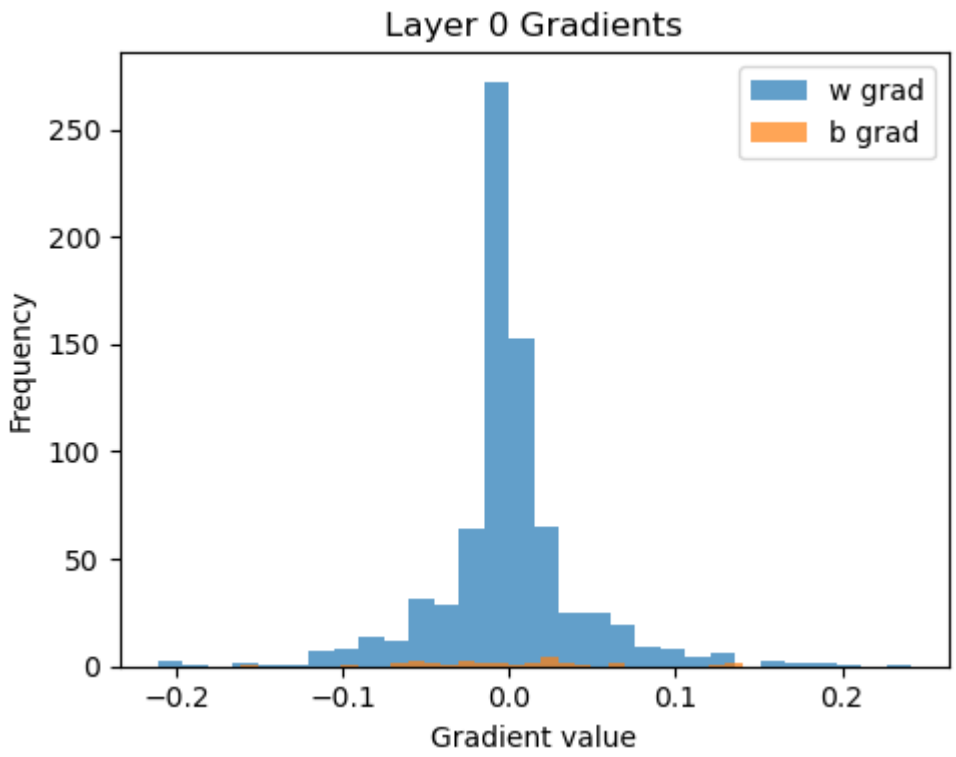


Distribusi bobot (Layer 0) tersebar secara normal di sekitar angka nol. Gradien memuncak dengan aman dan merata, menandakan bahwa satu-satunya hidden layer yang ada menerima sinyal error secara langsung dan utuh dari output, tanpa ada reduksi informasi dari layer-layer di depannya.

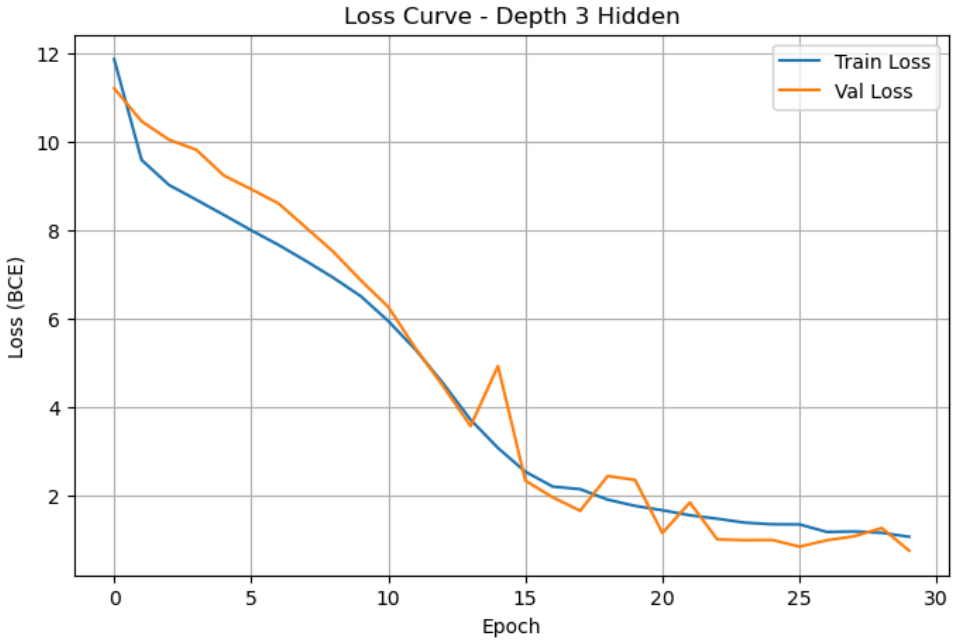


Model dengan 2 hidden layer memperlihatkan penurunan loss yang identik mulusnya dengan *Depth 1*, namun mampu mendorong nilai loss akhir sedikit lebih rendah, yang selaras dengan pencapaian akurasi terbaiknya (0.69) di tabel metrik. Model ini berhasil menyeimbangkan antara kemampuan generalisasi dan kemampuan abstraksi fitur.

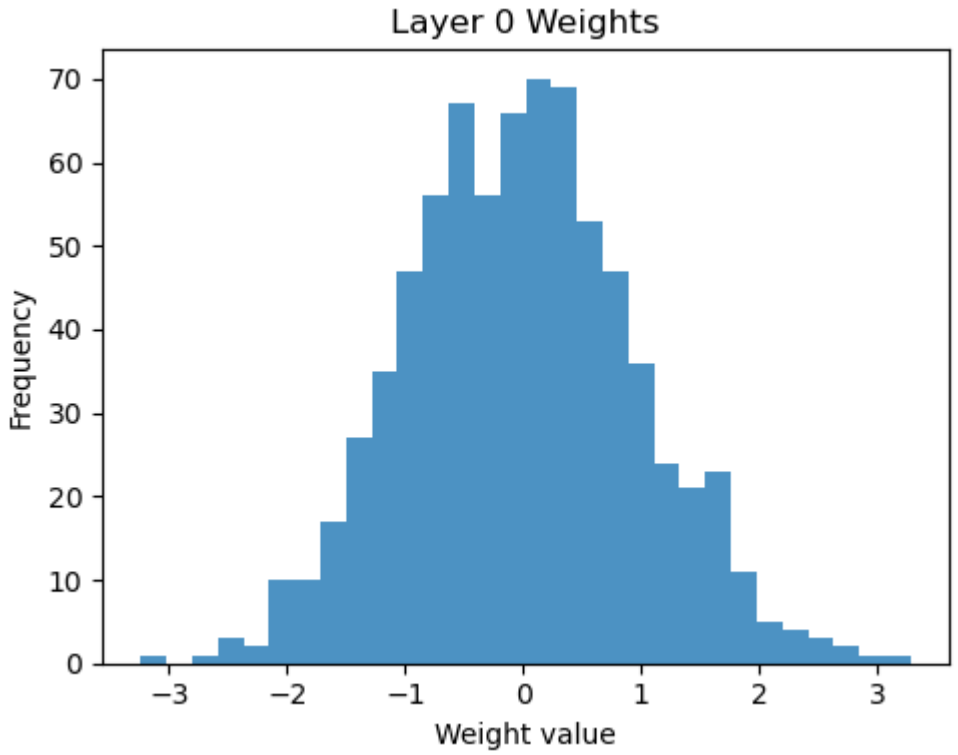


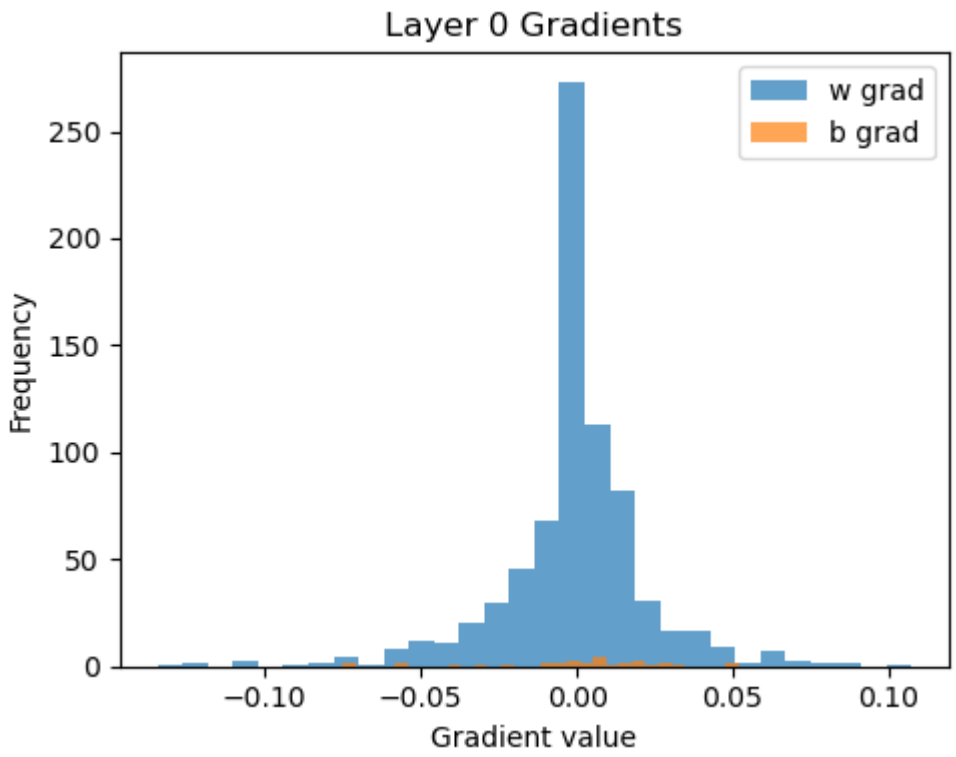


Histogram bobot pada Layer 0 menunjukkan distribusi yang masih menyerupai kurva normal dengan variansi yang relatif besar. Distribusi gradien memperlihatkan nilai yang bervariasi dan tidak tereduksi secara signifikan, yang mengindikasikan bahwa gradien dari proses backpropagation masih dapat dipropagasikan hingga layer awal jaringan. Kondisi ini memungkinkan Layer 0 tetap menerima pembaruan parameter yang efektif dalam proses pembelajaran representasi fitur awal dari dataset.



Kurva loss pada *Depth 3* menunjukkan indikasi ketidakstabilan, yang terlihat dari lonjakan pada validation loss sekitar epoch ke-14. Pada jaringan dengan kedalaman lebih tinggi, proses pembaruan parameter antar layer dapat mengalami ketidakselarasan selama optimasi berbasis batch, sehingga menyebabkan fluktuasi sementara pada nilai loss sebelum mekanisme optimisasi kembali menurunkannya.





Pada *Layer 0* (paling awal), distribusi gradien memusat secara ekstrem di angka nol (jauh lebih tajam dibanding *Depth 1 dan 2*). Ini adalah indikasi dari masalah *Vanishing Gradient*; sinyal pembaruan dari fungsi *loss* semakin melemah dan mengecil saat dikalikan berulang kali mundur melewati 3 *layer*, menyebabkan *layer* awal menerima update yang sangat minim.

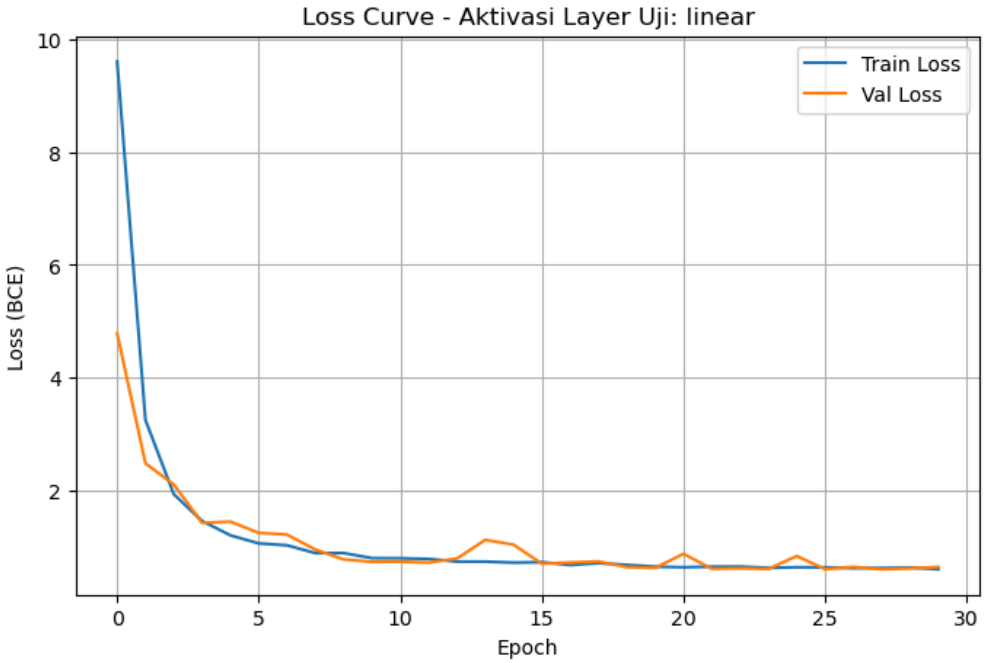
2.2.2. Pengaruh Fungsi Aktivasi

Metrik	*linear*	*Relu*	*Sigmoid*	*Tanh*
accuracy	0.67	0.67	0.64	0.65
Macro F1	0.67	0.64	0.59	0.61
Weighted F1	0.67	0.66	0.62	0.64
Placed (1) Precision	0.80	0.71	0.67	0.68
Placed (1) Recall	0.60	0.77	0.81	0.79
Not Placed (0) Precision	0.56	0.59	0.57	0.57
Not Placed (0) Recall	0.77	0.51	0.38	0.43

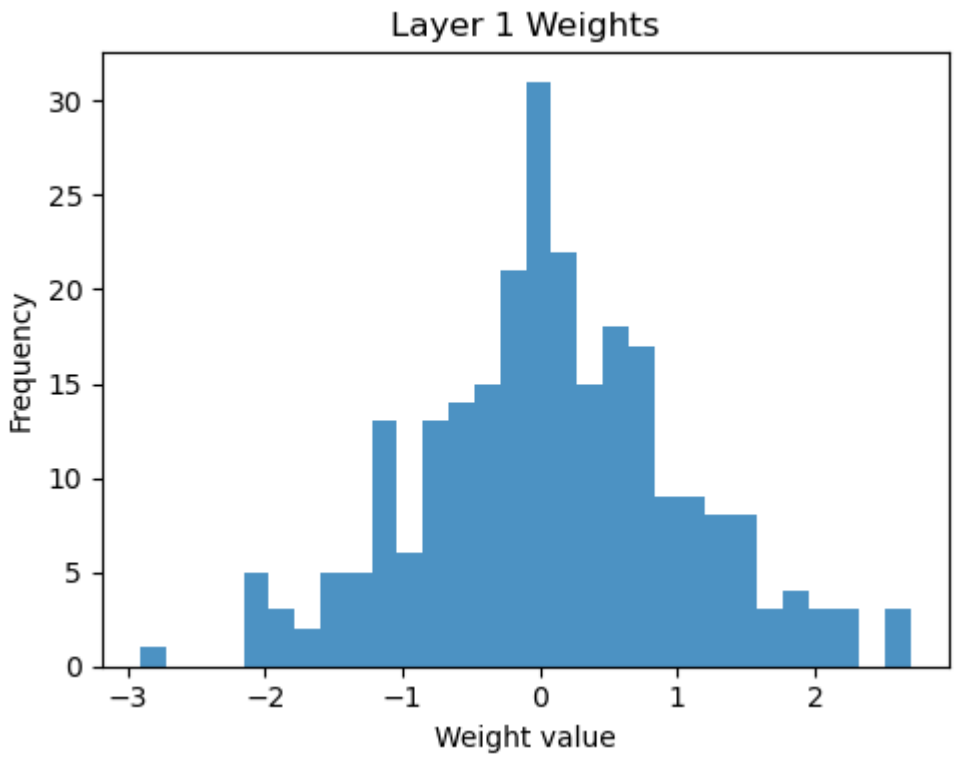
Eksperimen yang dilakukan dengan menyisipkan berbagai fungsi aktivasi pada layer pengujian (Layer 1) menunjukkan bahwa penggunaan fungsi aktivasi ReLU dan Linear menghasilkan nilai akurasi tertinggi, yaitu sebesar 0,67. Sebaliknya, fungsi aktivasi Sigmoid menunjukkan performa paling rendah dengan nilai akurasi 0,64 serta nilai Macro F1-score yang relatif rendah, yaitu 0,59.

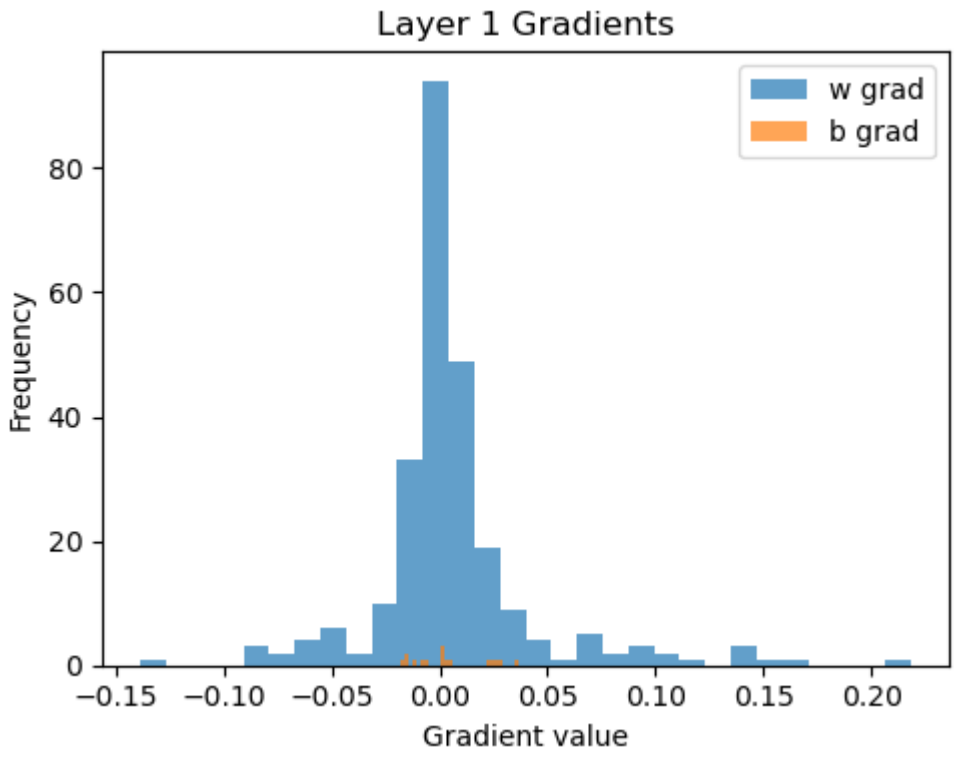
Meskipun fungsi aktivasi Linear menunjukkan performa yang cukup baik berdasarkan metrik evaluasi pada eksperimen ini, kondisi tersebut kemungkinan dipengaruhi oleh karakteristik dataset yang masih memiliki hubungan linier yang cukup kuat. Namun demikian, secara teoritis penggunaan fungsi aktivasi Linear pada *neural network* dapat mengurangi kemampuan model dalam mempelajari representasi nonlinier, sehingga berpotensi menurunkan efektivitas kedalaman jaringan.

Di sisi lain, fungsi aktivasi Sigmoid menunjukkan kinerja terendah yang dapat dikaitkan dengan karakteristik fungsinya yang cenderung menekan atau mengompresi nilai sinyal ke dalam rentang terbatas. Sifat ini berpotensi menyebabkan terjadinya vanishing gradient, sehingga proses pembelajaran menjadi kurang efektif dan berdampak pada penurunan performa model.

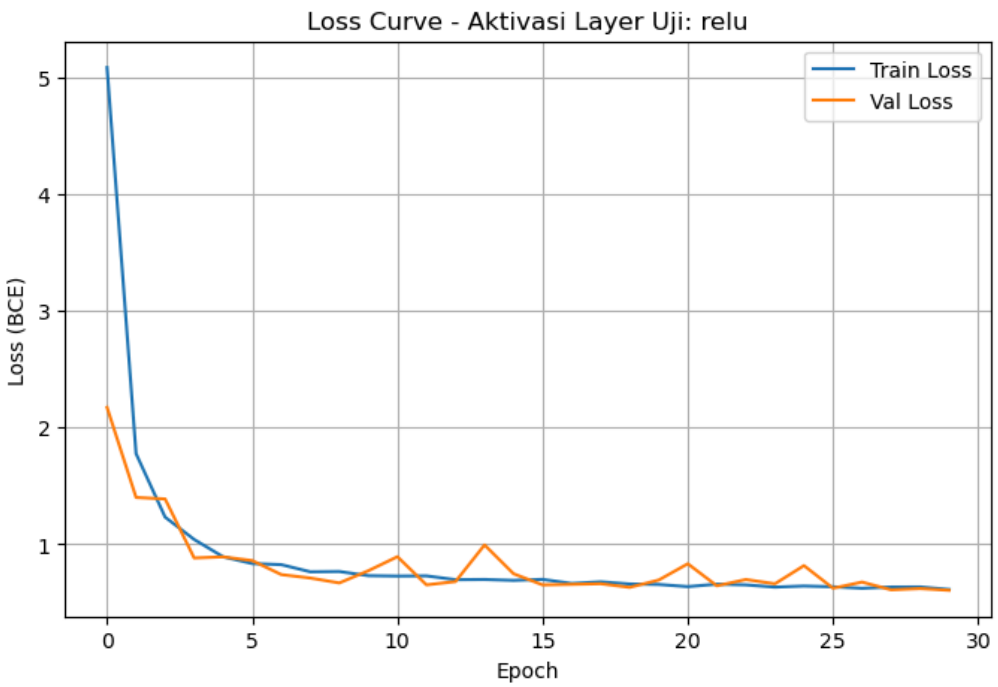


Kurva loss dengan aktivasi Linear jatuh sangat drastis dan hampir vertikal di epoch pertama, lalu segera mendatar dan stabil. Karena tidak ada "penekuk" non-linear (turunannya selalu konstan 1), aliran gradien bergerak masif dan tidak direm. Akibatnya model mengkonversi deep network menjadi regresi linier biasa yang cepat selesai namun terbatas kapasitas logikanya.

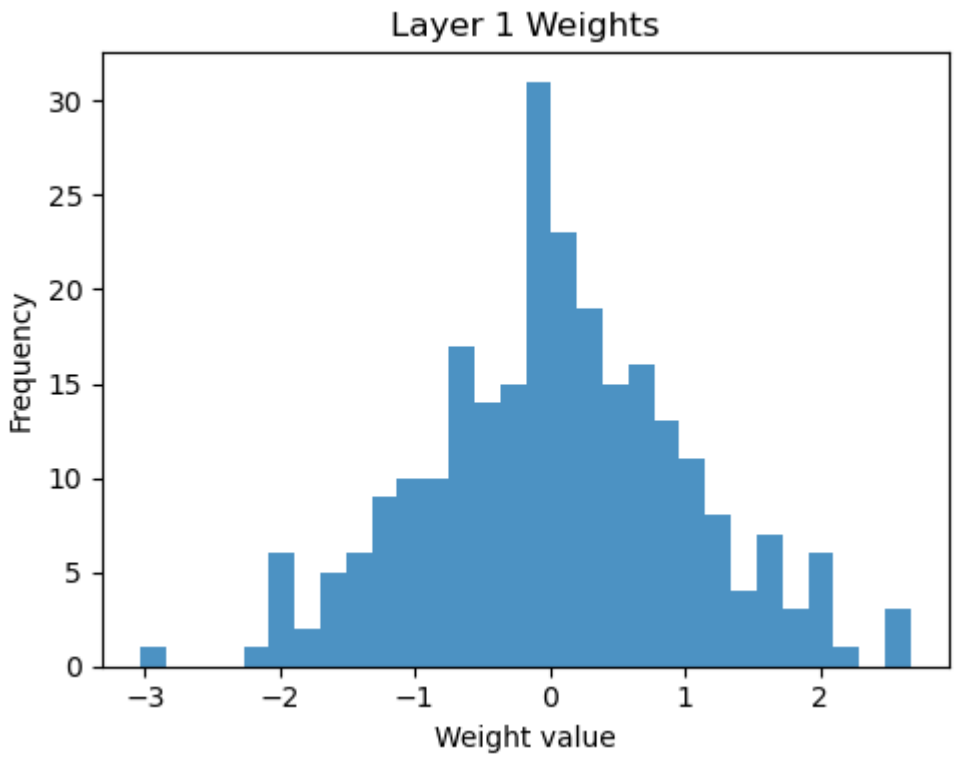


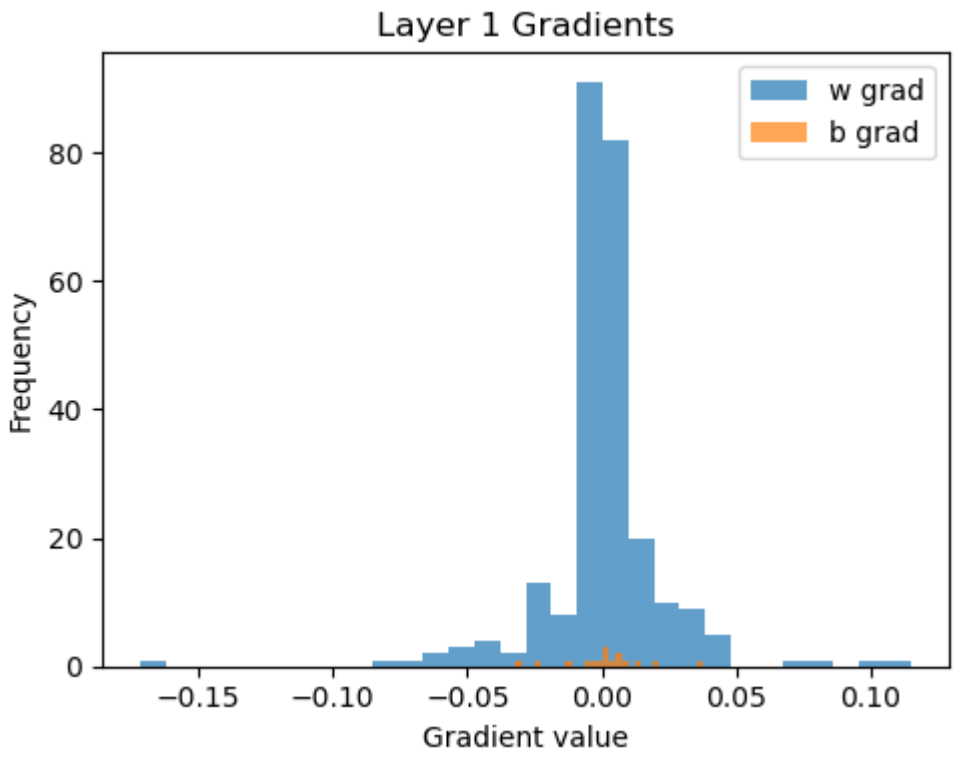


Distribusi bobot dan gradien tampak sangat lebar dan tersebar tanpa batasan ketat. Tanpa adanya pembatas non-linear (seperti pemotongan 0 pada ReLU atau penjepitan 0-1 pada Sigmoid), bobot bebas bergeser ke nilai-nilai yang jauh. Jika di-train lebih lama dengan learning rate besar, distribusi ini berisiko pecah menjadi exploding gradients.

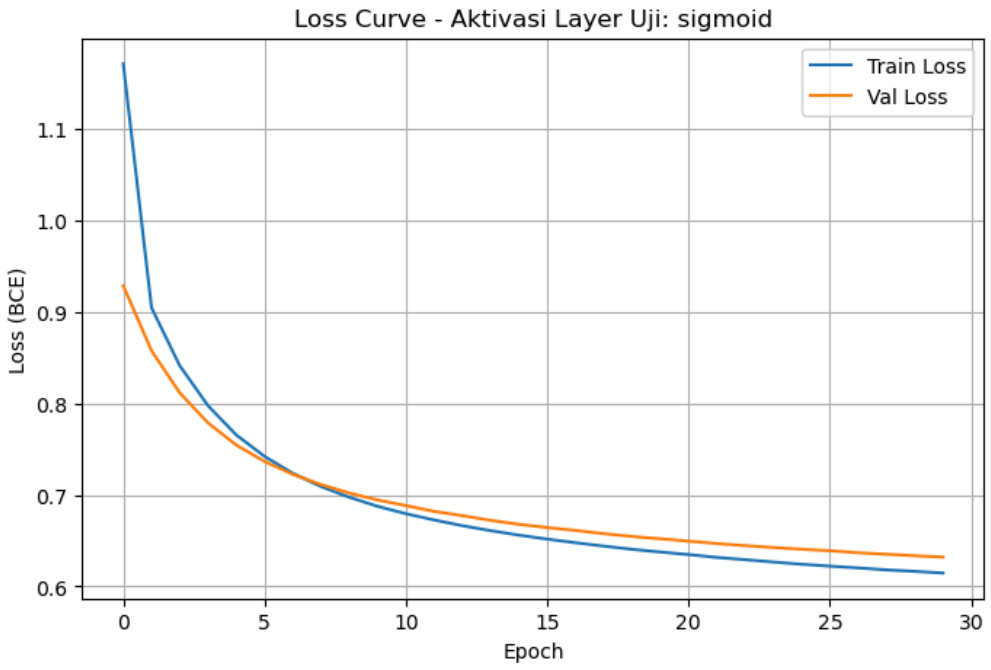


Kurva loss *ReLU* menunjukkan penurunan yang sangat agresif namun terkendali, dan mencapai nilai loss validasi yang lebih kompetitif di akhir *epoch*. Kurva ini mengonfirmasi mengapa *ReLU* menjadi standar industri modern; *ReLU* mampu menurunkan loss dengan cepat tanpa mengorbankan stabilitas karena ia tidak pernah menekan sinyal bernilai positif.





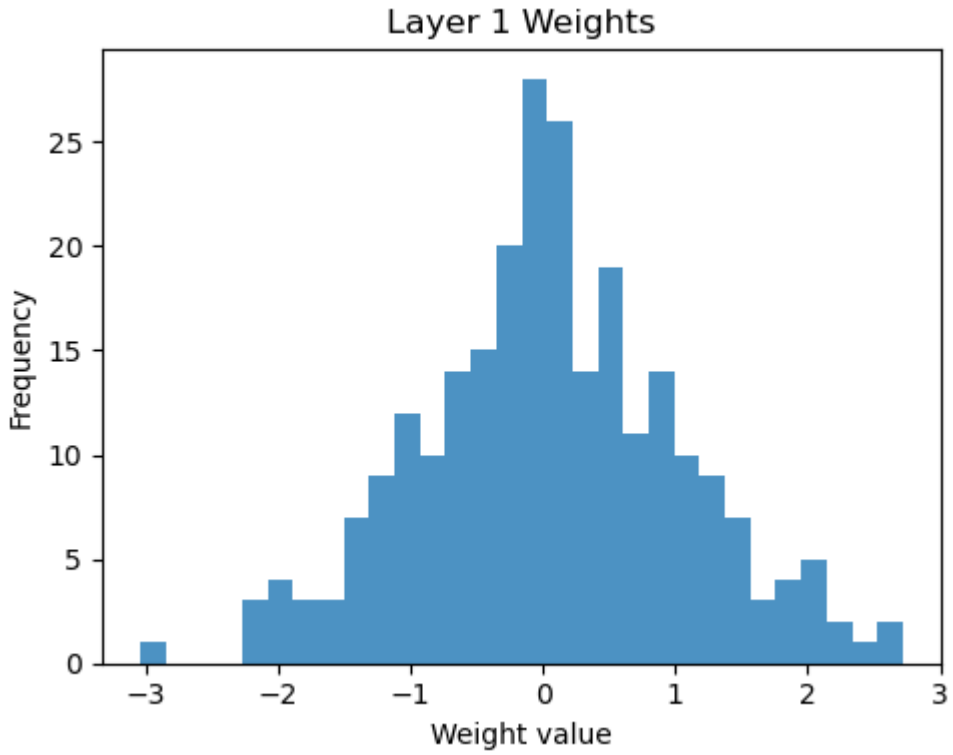
Distribusi bobot membentuk kurva lonceng yang baik. Pada distribusi gradien, sebaran nilainya lebih merata dan tidak memusat mati di angka nol. Bagian negatif dari ReLU memang menghasilkan nol, namun aliran gradien dari nilai positif terus menjaga network tetap hidup dan belajar, mencegah masalah layer yang membeku.

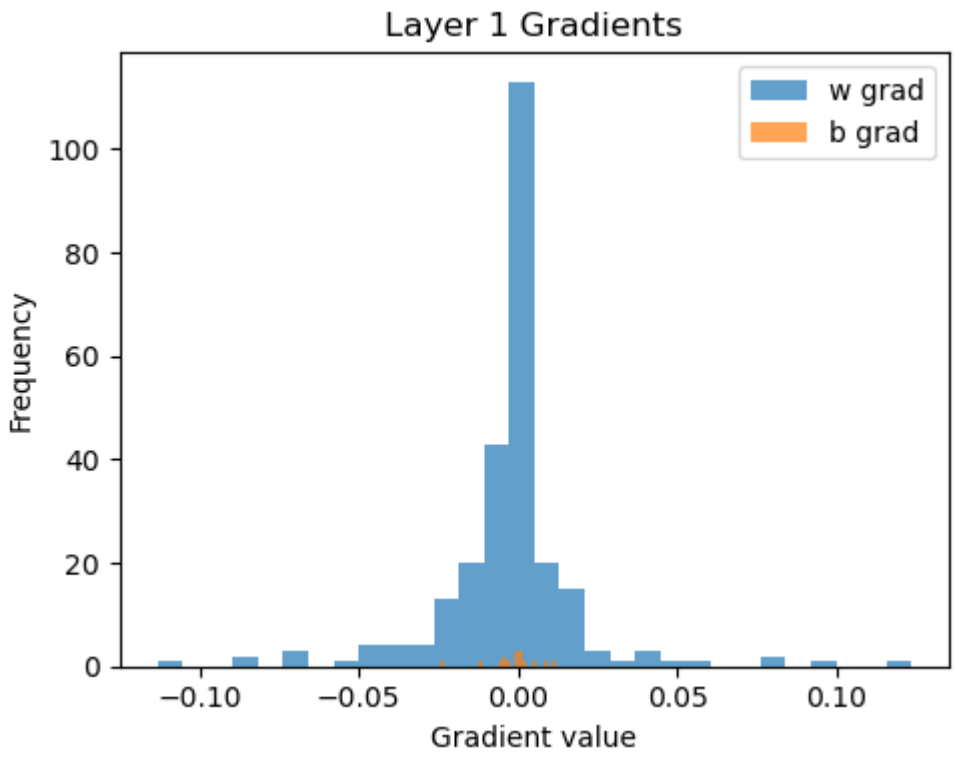


Berbeda secara signifikan dibandingkan fungsi aktivasi lainnya, kurva loss pada Sigmoid menunjukkan pola penurunan yang sangat gradual, cenderung landai, dan hampir mendatar sepanjang proses pelatihan. Konvergensi yang terjadi berjalan sangat lambat sehingga nilai loss akhir tetap relatif tinggi, yakni sekitar 0.6. Fenomena ini merepresentasikan secara jelas kondisi optimization stagnation yang disebabkan oleh gradient squashing atau kompresi gradien pada fungsi aktivasi Sigmoid.

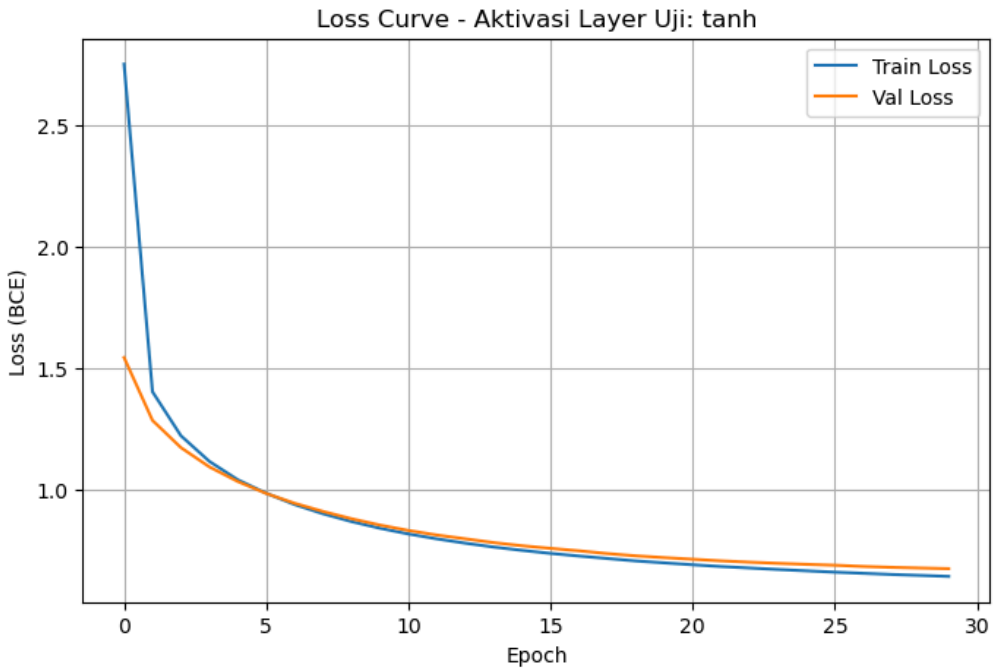
Secara matematis, karena rentang keluaran Sigmoid terbatas pada interval (0, 1), nilai turunan (gradient) yang dihasilkan menjadi sangat kecil, terutama ketika neuron berada pada daerah saturasi (nilai input sangat positif atau sangat negatif). Akibatnya, selama proses backpropagation, gradien yang mengalir ke lapisan sebelumnya mengalami *attenuation* yang signifikan (*vanishing gradient effect*), sehingga pembaruan parameter melalui algoritma optimasi (misalnya gradient descent) menjadi sangat minimal pada setiap iterasi.

Implikasinya, dinamika pembelajaran *neural network* menjadi tidak efisien karena magnitude pembaruan bobot (*weight update*) terlalu kecil untuk menghasilkan penurunan loss yang substansial, sehingga model mengalami keterlambatan konvergensi dan performa pelatihan yang suboptimal.

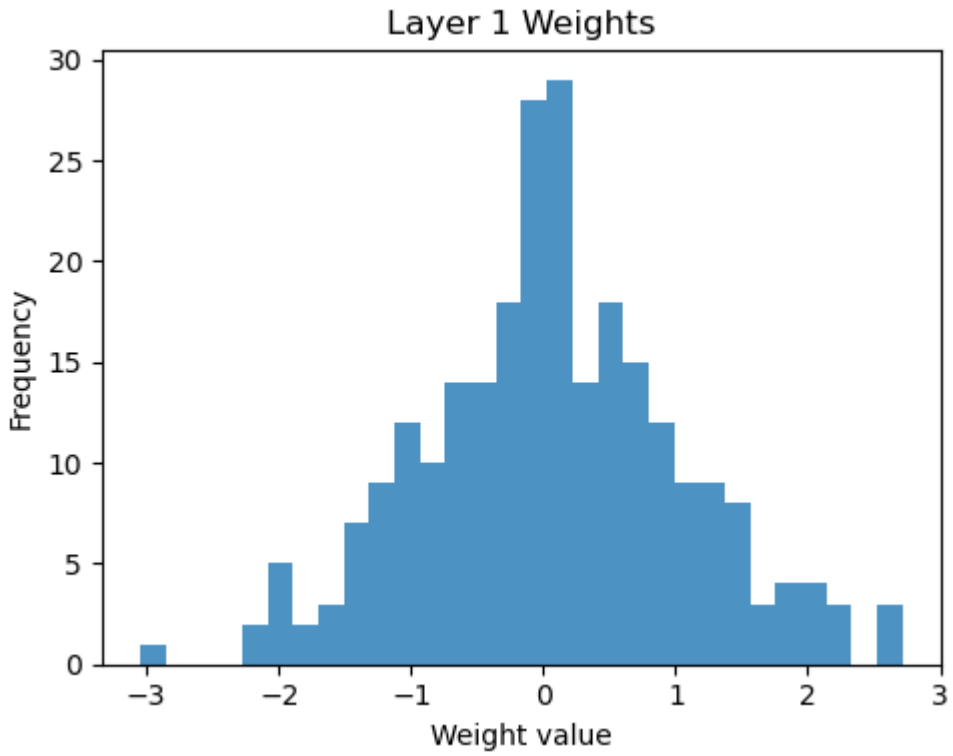


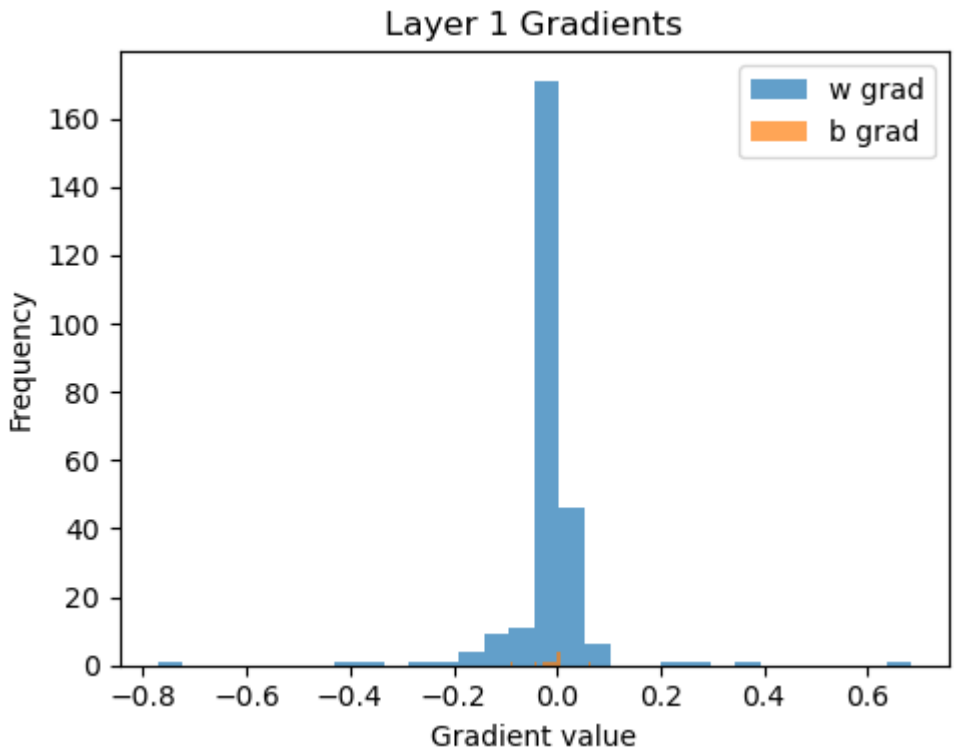


Histogram gradien untuk Sigmoid memperlihatkan frekuensi *spike* yang menumpuk ekstrem persis di angka nol. Turunan Sigmoid maksimum hanyalah 0.25, yang berarti setiap kali gradien melewati layer Sigmoid, nilainya menyusut 75%. Akibatnya, bobot tidak bergeser, model tidak belajar, dan distribusi terekam "mati" di tengah.



Kurva loss Tanh bergerak lebih cepat menurun dibandingkan Sigmoid, namun sedikit lebih lambat dari ReLU. Hal ini terjadi karena Tanh memiliki jangkauan nilai nol-sentris (-1 hingga 1) dan turunan tertingginya adalah 1 (bukan 0.25 seperti Sigmoid), sehingga mereduksi error dengan lebih efisien tanpa mengalami kelumpuhan total di awal.



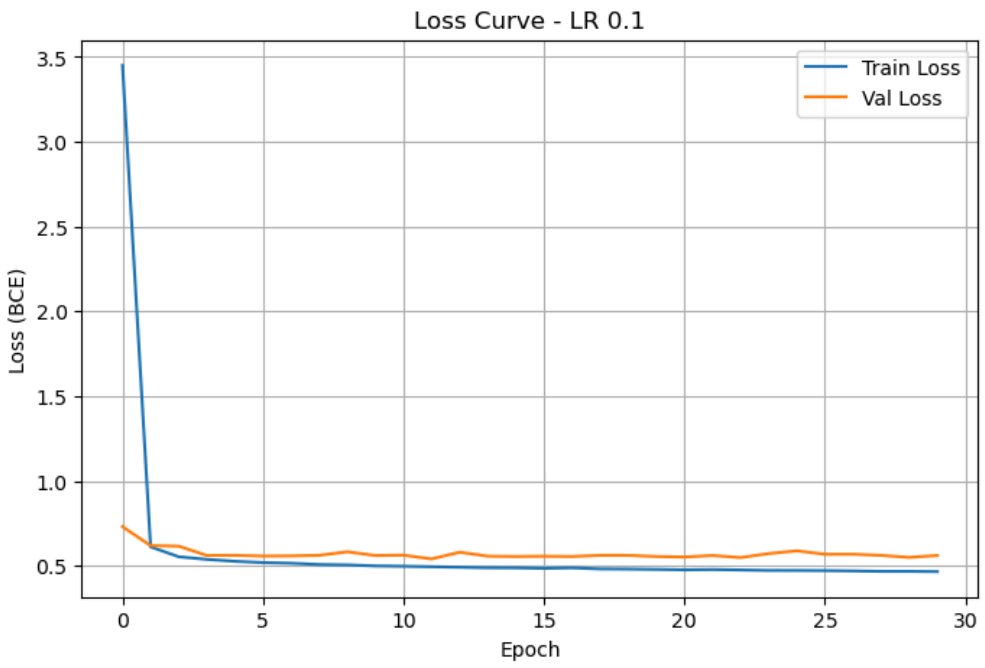


Distribusi bobot Tanh terjaga dalam rentang yang ketat, dan gradiennya memiliki penyebaran yang lebih luas di kedua sisi nol dibandingkan Sigmoid. Meskipun pada akhirnya Tanh juga rentan terhadap vanishing gradient di area saturasi (ujung kurva), sentralisasi nol-nya membantu gradien mengalir lebih bebas selama fase awal pelatihan.

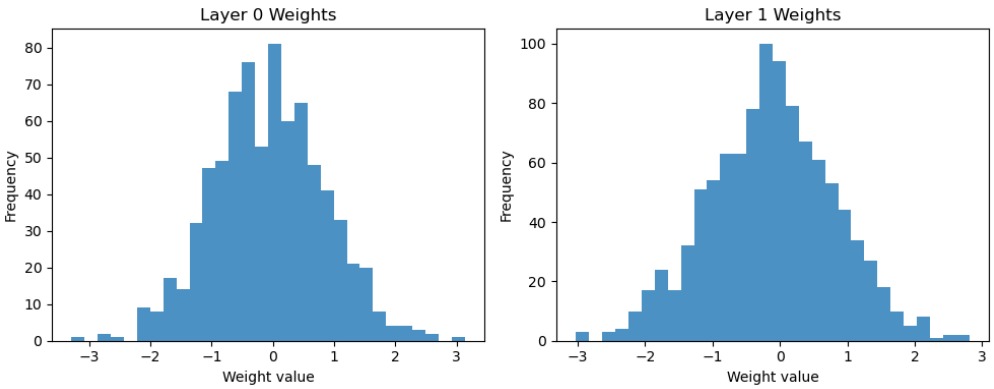
2.2.3. Pengaruh Learning Rate

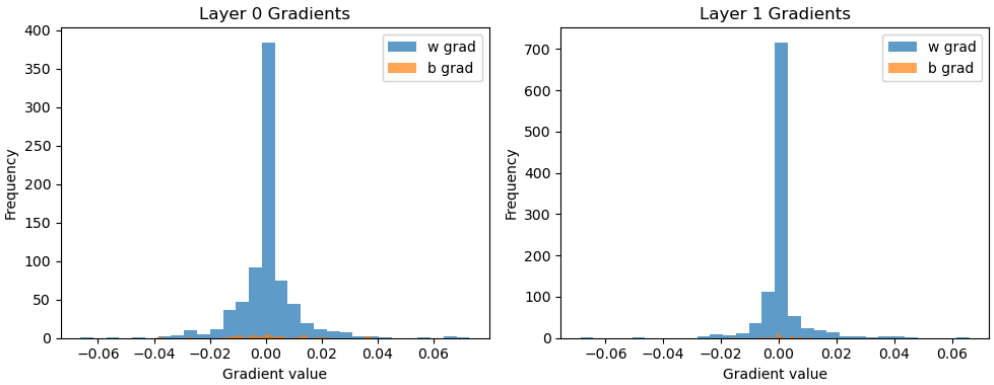
Metrik	*learning rate 0.1*	*learning rate 0.01*	*learning rate 0.001*
accuracy	0.72	0.69	0.69
Macro F1	0.69	0.63	0.66
Weighted F1	0.71	0.66	0.68
Placed (1) Precision	0.73	0.68	0.71
Placed (1) Recall	0.85	0.91	0.81
Not Placed (0) Precision	0.69	0.73	0.63
Not Placed (0) Recall	0.52	0.36	0.51

Berdasarkan tabel, Learning Rate (LR) 0.1 memberikan Akurasi tertinggi (0.72) dibandingkan LR 0.01 (0.69) dan LR 0.001 (0.69). Meskipun secara angka LR terbesar tampak mendominasi pada data validasi akhir, grafik loss dan distribusi gradien akan membuktikan bahwa LR ekstrem ini menyimpan instabilitas besar dan sangat berisiko gagal konvergen secara konsisten.

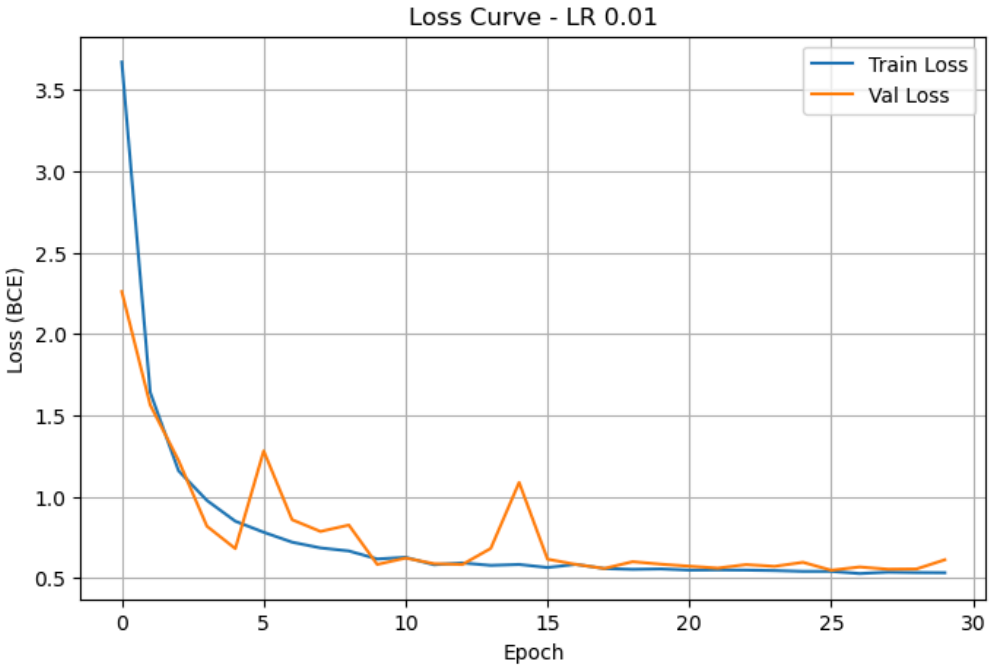


Kurva loss pada konfigurasi learning rate (LR) 0.1 menunjukkan penurunan yang sangat tajam pada epoch pertama, diikuti oleh pola konvergensi yang cenderung horizontal dengan osilasi kecil hingga akhir proses pelatihan. Nilai learning rate yang sangat besar menyebabkan algoritma optimisasi, misalnya gradient descent atau variannya, melakukan pembaruan parameter dengan magnitude yang sangat tinggi, sehingga dalam satu iterasi awal model mampu melompati wilayah dengan loss tinggi secara drastis. Namun, ukuran *step size* yang terlalu besar ini juga memicu fenomena *overshooting* terhadap titik minimum, pembaruan bobot secara berulang melewati area lembah dari global optimum pada permukaan fungsi loss. Akibatnya, proses optimisasi tidak benar-benar mencapai titik konvergensi yang stabil, melainkan berosilasi di sekitar wilayah minimum dengan fluktuasi kecil, yang tercermin dari kurva loss yang relatif mendatar namun tidak sepenuhnya stabil sepanjang sisa epoch pelatihan.



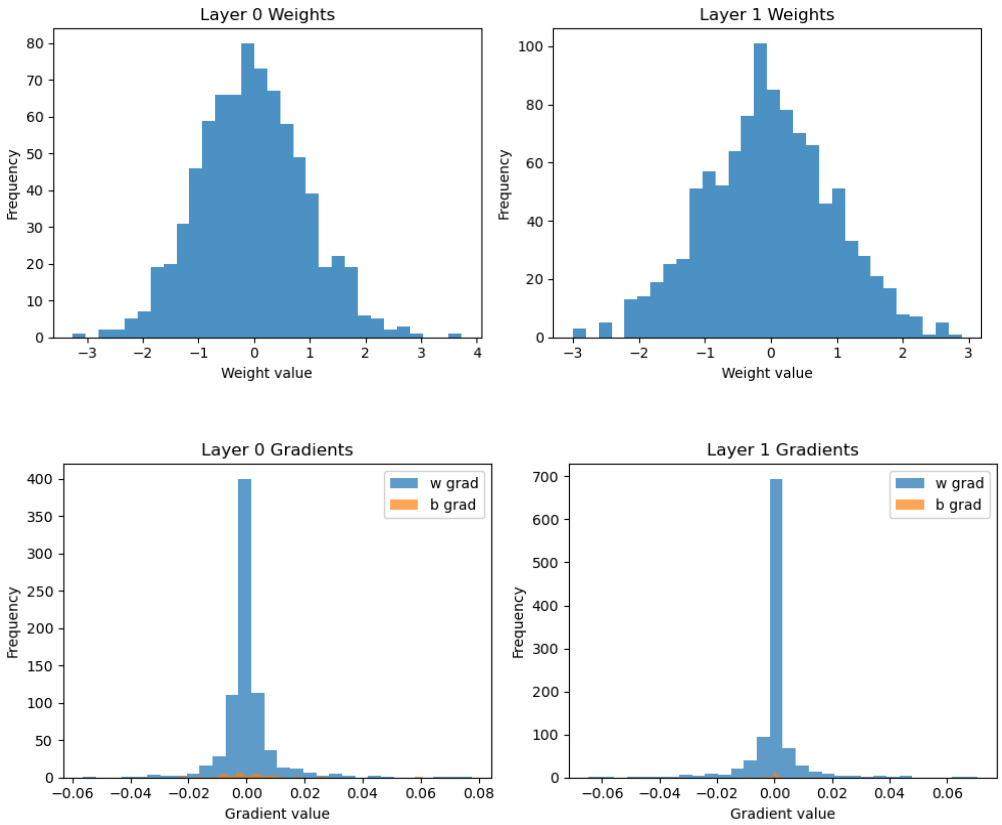


Histogram distribusi bobot dan gradien memperlihatkan karakteristik sebaran yang sangat kasar dan *noisy*. Pola ini mengindikasikan bahwa setiap langkah pembaruan parameter yang dilakukan oleh algoritma optimasi menghasilkan perubahan bobot dengan amplitudo yang sangat tinggi, sehingga pembaruan tersebut bersifat agresif dan tidak terkontrol. Akibatnya, distribusi parameter tidak menunjukkan pola penyempurnaan yang stabil atau gradual, melainkan memperlihatkan dinamika yang tidak terstruktur dan fluktuatif di ruang parameter. Kondisi ini mencerminkan bahwa jaringan saraf tidak mengalami proses refinement parameter secara bertahap, tetapi justru mengalami pergerakan parameter yang tidak stabil, di mana bobot terus terdorong secara drastis ke berbagai arah akibat gradien yang berfluktuasi besar selama proses backpropagation.

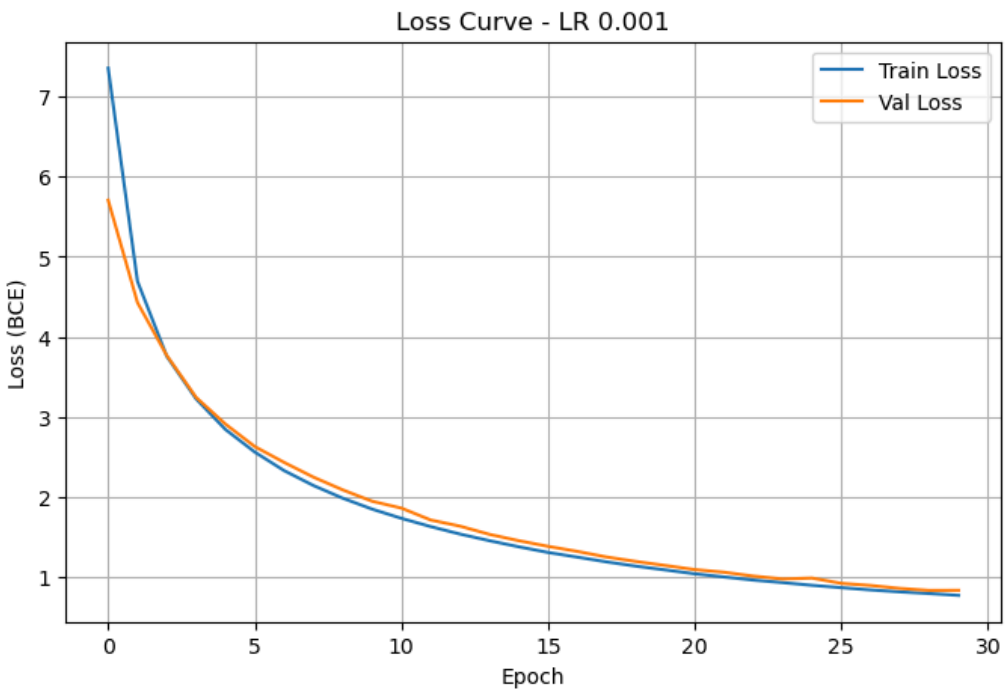


Kurva loss untuk LR 0.01 adalah contoh kurva pelatihan yang paling baik di eksperimen ini. Loss turun secara bertahap namun pasti, menunjukkan beberapa tonjolan kecil penyesuaian yang wajar saat model beradaptasi

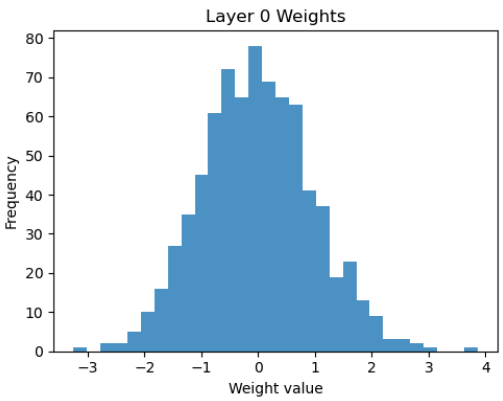
dengan batch yang sulit, dan akhirnya menemukan garis ekuilibrium yang baik. Ini adalah laju penyesuaian (step size) yang paling ideal.

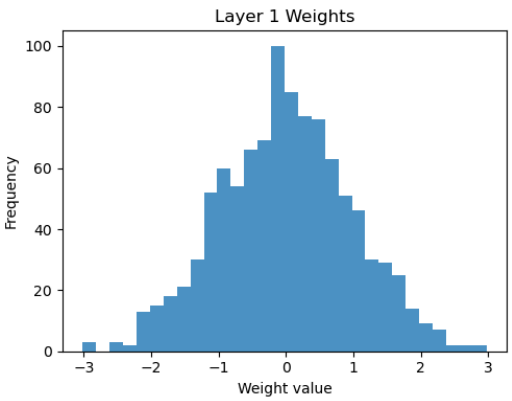


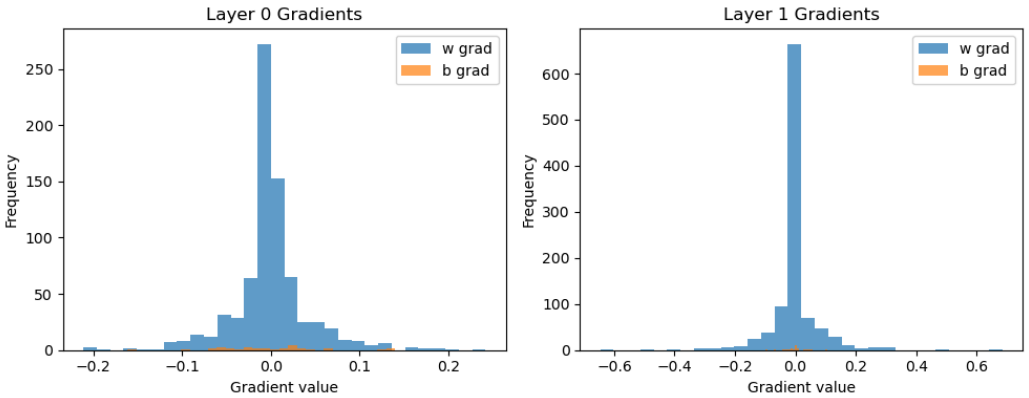
Histogram bobot menunjukkan distribusi normal yang sangat rapi dan proporsional. Distribusi gradien terkendali dengan baik dan menguncup wajar menuju nol. Ini menandakan bahwa pembaruan bobot terjadi dengan kehati-hatian matematis yang tepat, tidak terlalu berlebihan, dan tidak terlalu stagnan.



Kurva loss pada konfigurasi learning rate (LR) 0.001 menunjukkan pola penurunan yang sangat gradual, halus, dan stabil sepanjang proses pelatihan, tanpa indikasi osilasi maupun fluktuasi yang berarti pada setiap epoch. Meskipun dinamika konvergensi ini tampak sangat stabil dan secara visual terlihat optimal, pada epoch ke-30 nilai loss masih berada pada tingkat yang relatif tinggi, yang mengindikasikan bahwa model belum mencapai kondisi konvergensi menuju minimum fungsi kerugian secara optimal. Nilai learning rate yang terlalu kecil menyebabkan magnitudo pembaruan parameter pada setiap iterasi menjadi sangat minimal, sehingga algoritma optimasi bergerak sangat lambat dalam menelusuri permukaan fungsi loss. Akibatnya, proses pencarian menuju titik minimum global atau lokal berlangsung dengan efisiensi yang rendah dan memerlukan jumlah epoch yang jauh lebih besar untuk mencapai konvergensi yang memadai, sehingga secara praktis membutuhkan tambahan ratusan iterasi pelatihan agar model dapat mencapai minimum pada lanskap optimisasi.







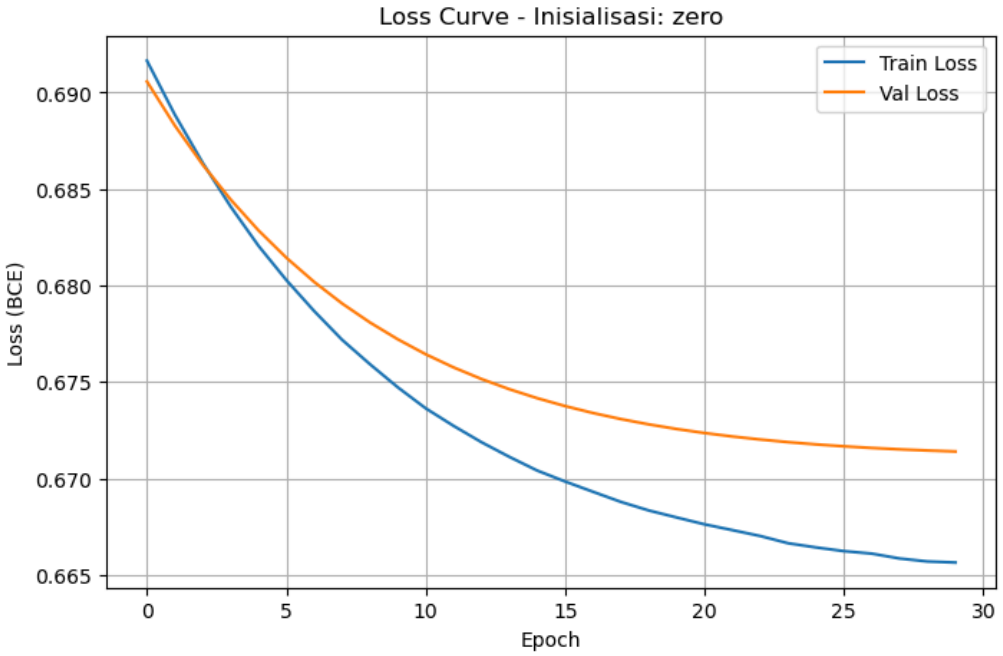
Distribusi gradien pada histogram LR 0.001 menjadi sangat tajam di angka nol absolut. Sinyal gradien yang sejatinya sehat dikalikan dengan LR (0.001) yang sangat kecil, membuat hasil pergeseran bobot nyaris tidak ada. Akibatnya, distribusi bobot sebagian besar hanya diam mematung pada kondisi inisialisasi awal mereka.

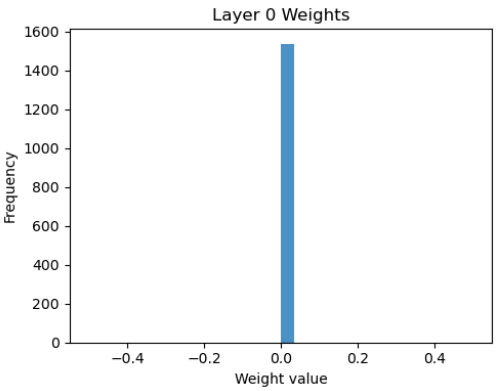
2.2.4. Pengaruh Metode Inisialisasi Bobot

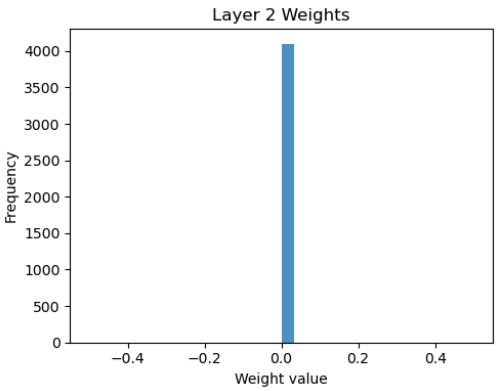
Metrik	*zero*	*uniform*	*normal*	*xavier*	*he*
accuracy	0.60	0.60	0.72	0.74	0.73
Macro F1	0.38	0.38	0.70	0.71	0.70
Weighted F1	0.46	0.46	0.71	0.73	0.72
Placed (1) Precision	0.60	0.60	0.74	0.75	0.74
Placed (1) Recall	1.00	1.00	0.82	0.85	0.83
Not Placed (0) Precision	0.00	0.00	0.67	0.71	0.69
Not Placed (0) Recall	0.00	0.00	0.56	0.57	0.56

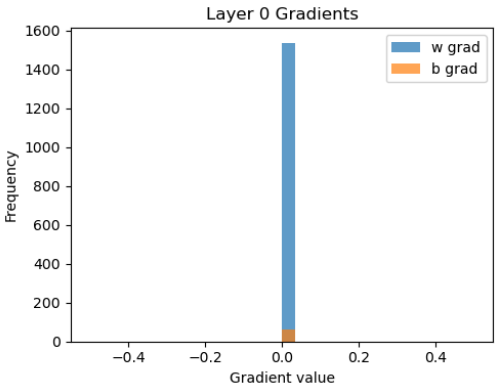
Tabel komparasi metode inisialisasi bobot memperlihatkan perbedaan performa yang sangat kontras antarpendekatan yang digunakan. Metode inisialisasi Zero (0) dan Uniform menunjukkan kinerja paling rendah dengan nilai akurasi hanya mencapai 0.60 dan Macro F1 sebesar 0.38, yang mengindikasikan kegagalan model dalam mempelajari representasi fitur secara efektif serta kecenderungan untuk memprediksi satu kelas dominan secara bias tanpa kemampuan diskriminatif yang memadai. Fenomena ini umumnya berkaitan dengan masalah *symmetry breaking failure* pada inisialisasi nol serta distribusi bobot awal yang kurang optimal pada pendekatan uniform

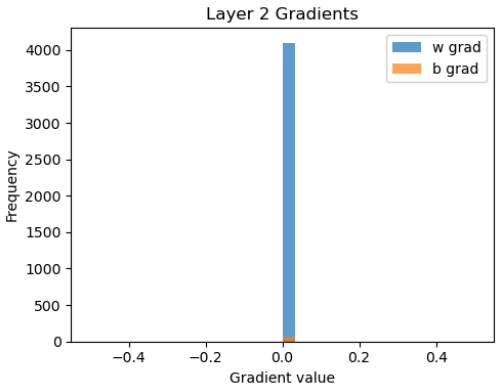
sederhana, sehingga proses gradient-based optimization tidak mampu mengembangkan representasi yang bermakna di setiap neuron. Sebaliknya, metode inisialisasi modern seperti *Xavier initialization* (akurasi 0.74) dan *He initialization* (akurasi 0.73) menunjukkan performa yang jauh lebih unggul karena dirancang untuk mempertahankan stabilitas variansi aktivasi dan gradien antar lapisan jaringan selama proses forward propagation dan backpropagation. Dengan menjaga skala distribusi sinyal tetap konsisten di seluruh lapisan, kedua metode ini memungkinkan proses pembelajaran berlangsung lebih stabil, mempercepat konvergensi, serta meningkatkan kemampuan model dalam mengekstraksi pola yang relevan dari data.

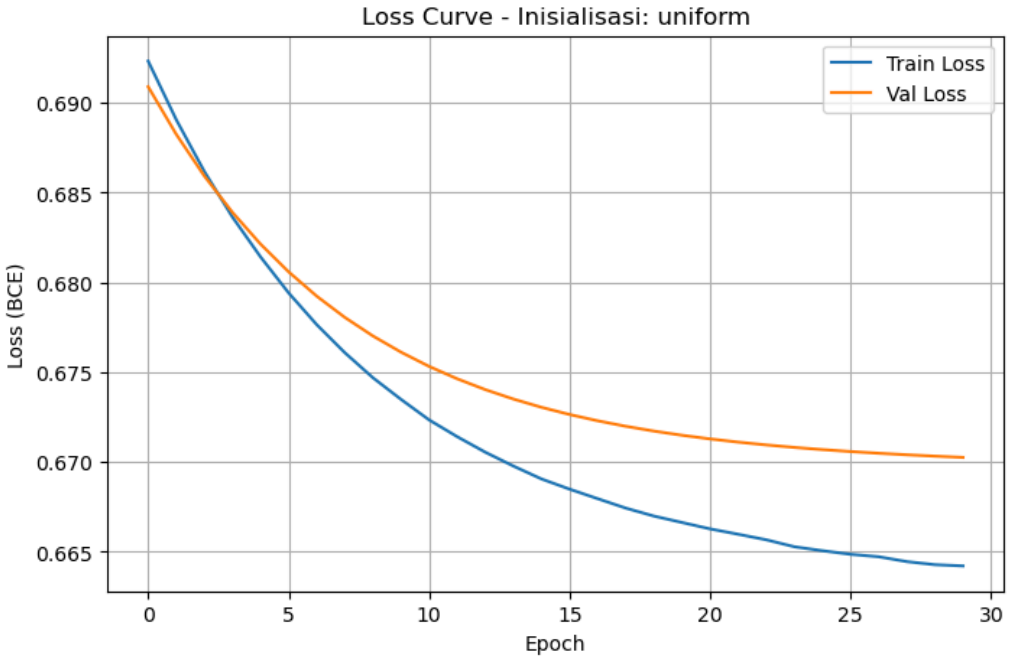


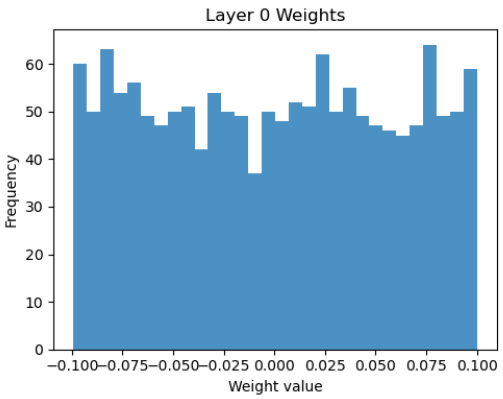


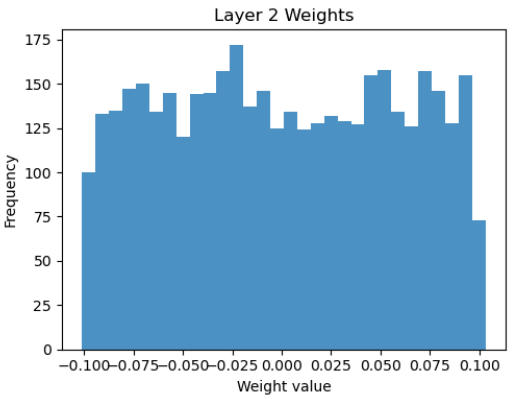


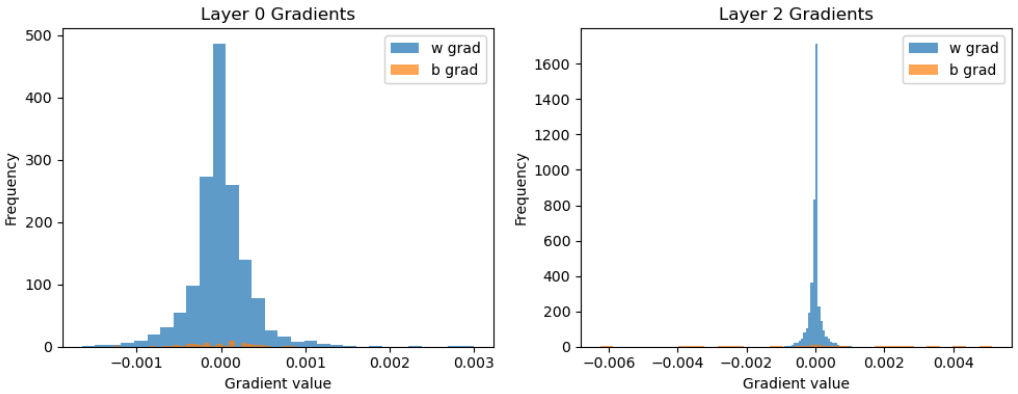

Layer 0 Gradients
w grad
b grad
Frequency
Gradient value

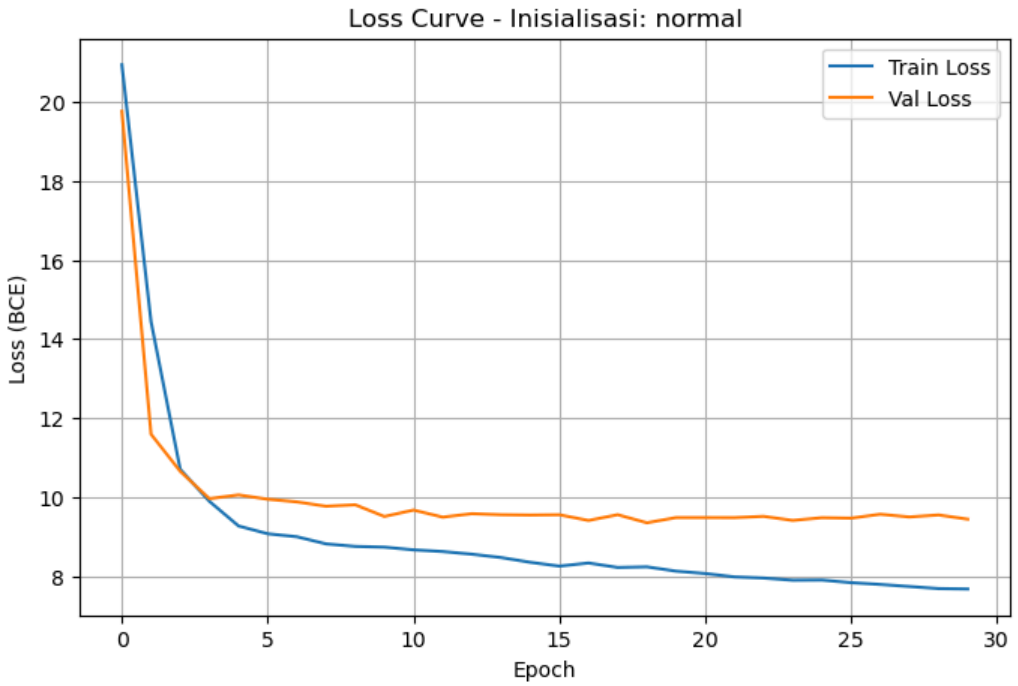

Layer 2 Gradients
w grad
b grad
Frequency
Gradient value

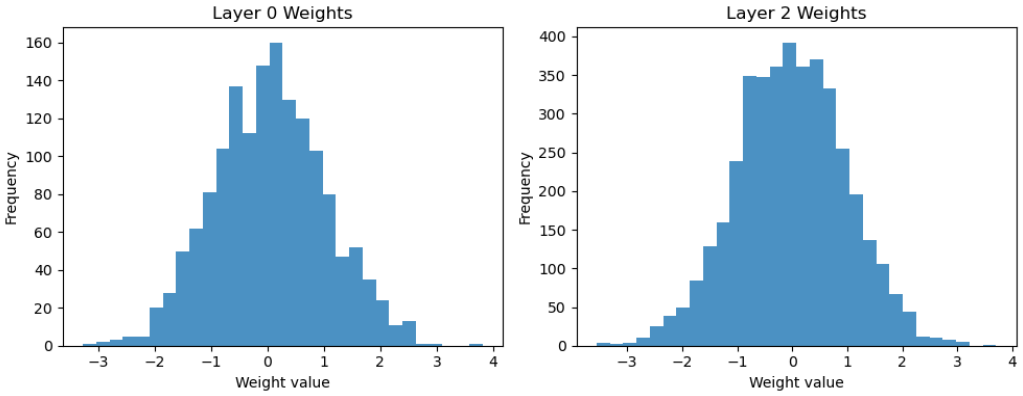

Loss Curve - Inisialisasi: uniform
Train Loss
Val Loss
Loss (BCE)
Epoch

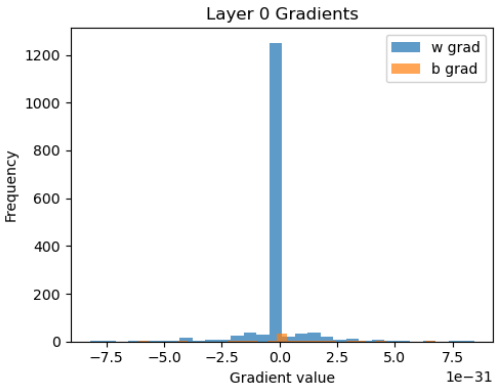

Layer 0 Weights
Frequency
Weight value

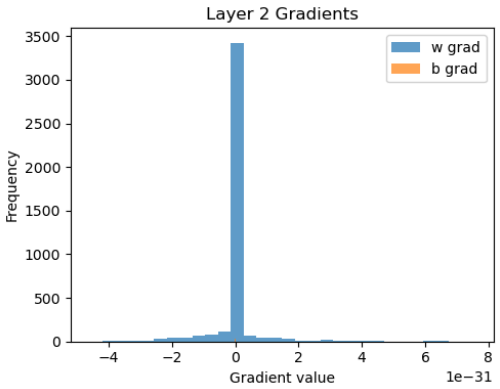

Layer 2 Weights
Frequency
Weight value

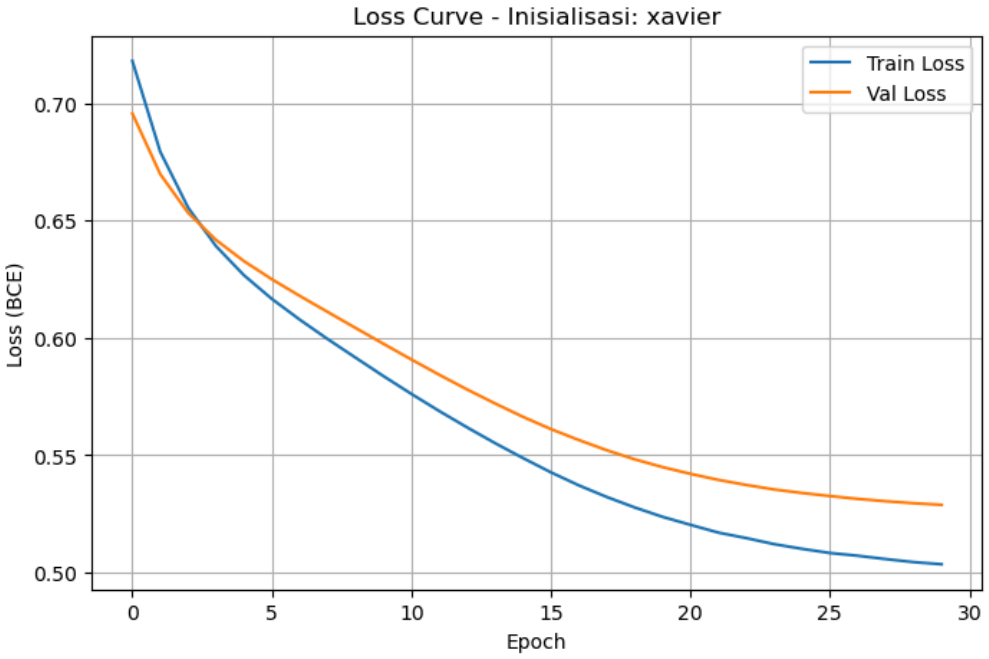

Layer 0 Gradients
w grad
b grad
Frequency
Gradient value
Layer 2 Gradients
w grad
b grad
Frequency
Gradient value

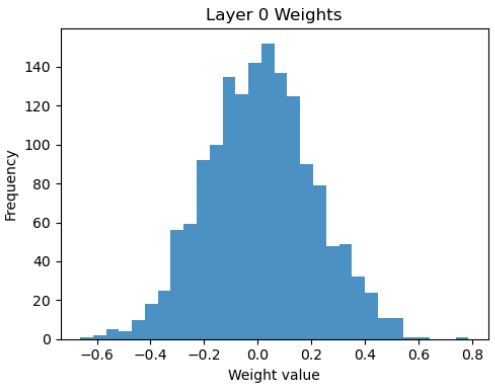

Loss Curve - Inisialisasi: normal
Train Loss
Val Loss
Loss (BCE)
Epoch

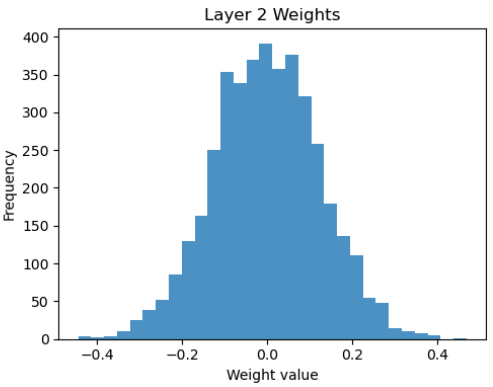

Layer 0 Weights
Frequency
Weight value
Layer 2 Weights
Frequency
Weight value

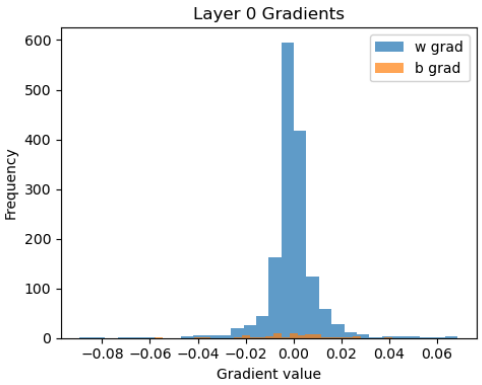

Layer 0 Gradients
w grad
b grad
Frequency
Gradient value
1e−31

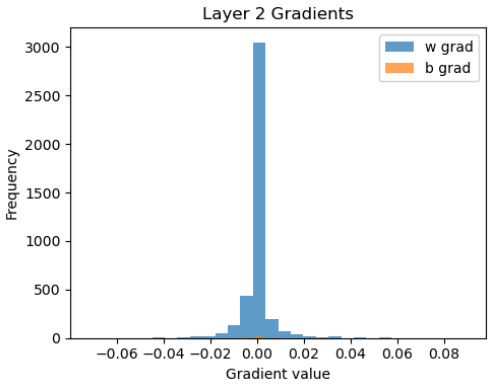

Layer 2 Gradients
w grad
b grad
Frequency
Gradient value
1e−31

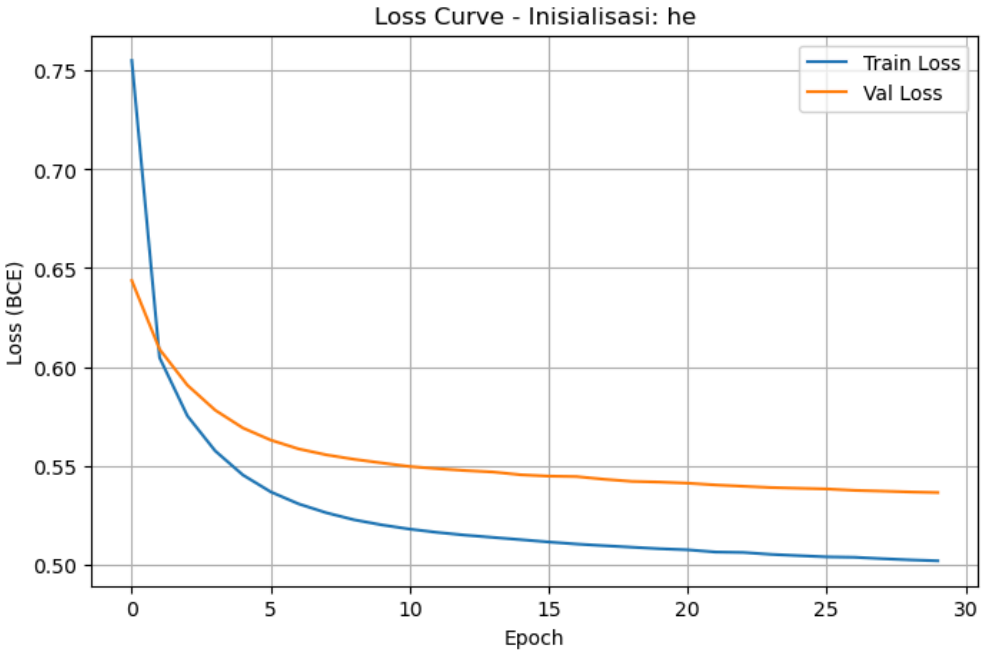

Loss Curve - Inisialisasi: xavier
Train Loss
Val Loss
Loss (BCE)
Epoch

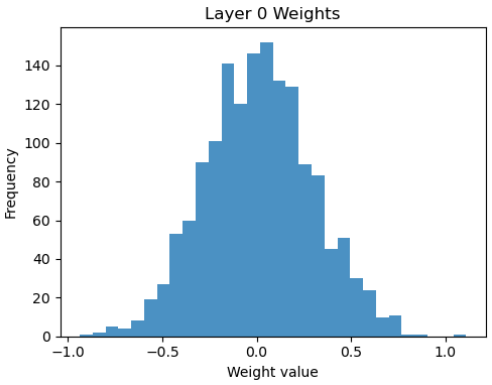

Layer 0 Weights
Frequency
Weight value

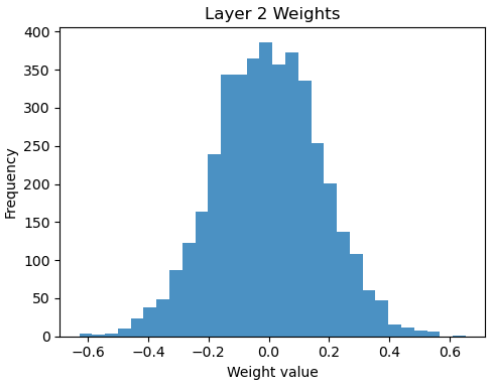

Layer 2 Weights
Frequency
Weight value

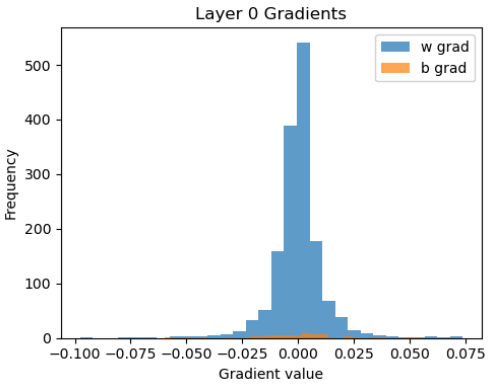

Layer 0 Gradients
w grad
b grad
Frequency
Gradient value

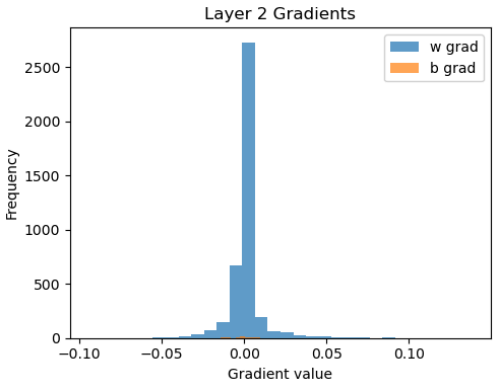

Layer 2 Gradients
w grad
b grad
Frequency
Gradient value


Loss Curve - Inisialisasi: he
Train Loss
Val Loss
Loss (BCE)
Epoch


Layer 0 Weights
Frequency
Weight value


Layer 2 Weights
Frequency
Weight value



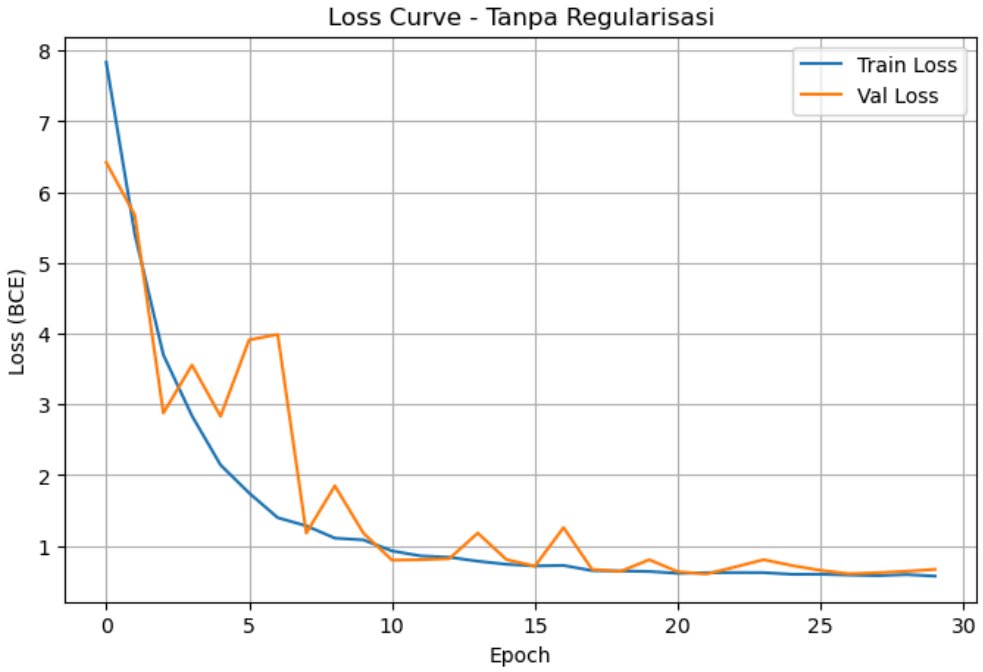


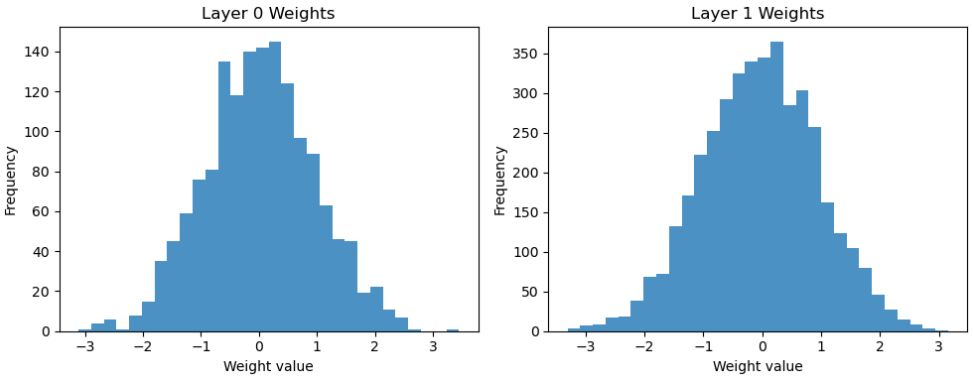
2.2.5. Pengaruh Regularisasi

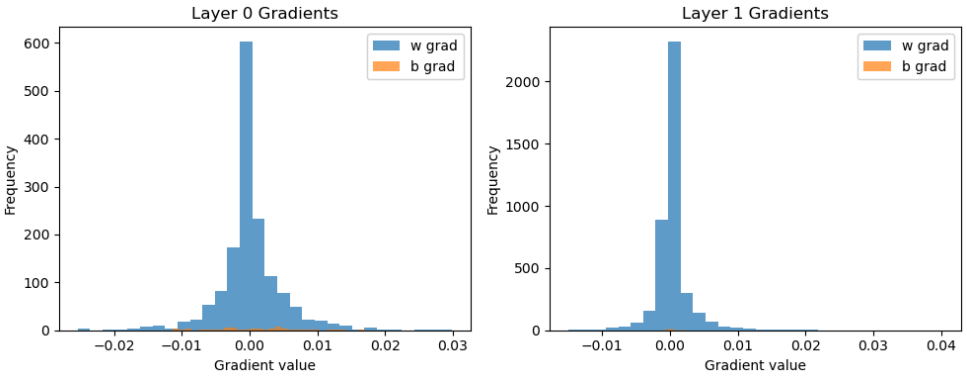
Metrik	Tanpa Regularisasi	Regularisasi L1 (0.001)	Regularisasi L2 (0.001)
accuracy	0.70	0.71	0.71
Macro F1	0.65	0.67	0.66
Weighted F1	0.68	0.69	0.69
Placed (1) Precision	0.70	0.71	0.71
Placed (1) Recall	0.88	0.88	0.89
Not Placed (0) Precision	0.70	0.71	0.72
Not Placed (0) Recall	0.42	0.45	0.43

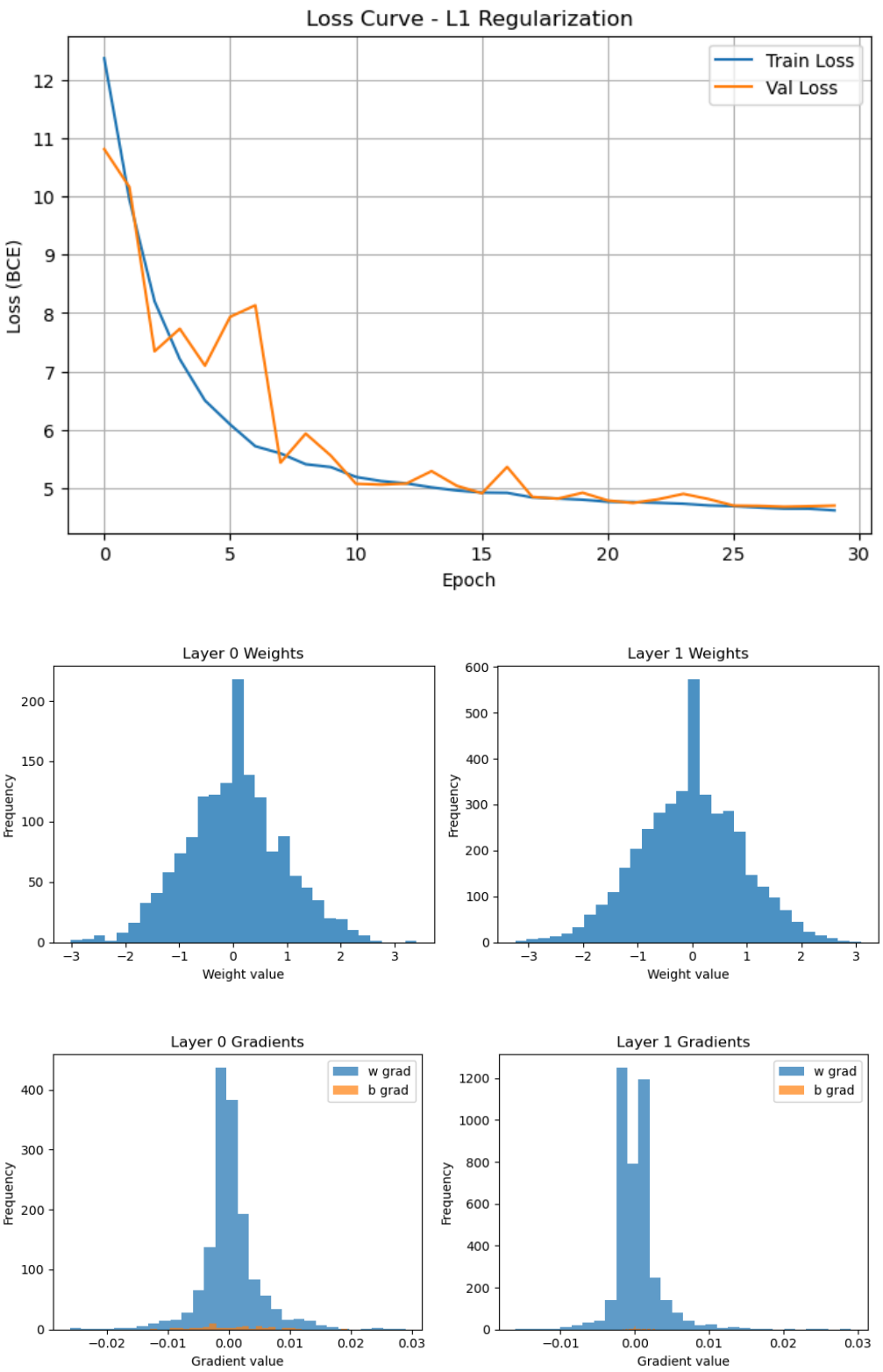
Penambahan mekanisme regularisasi pada proses pelatihan menunjukkan dampak yang konsisten terhadap peningkatan performa model, di mana konfigurasi tanpa regularisasi menghasilkan nilai akurasi sebesar 0.70, sedangkan model yang menerapkan regularisasi L1 maupun regularisasi L2 sama-sama mengalami peningkatan akurasi menjadi 0.71. Meskipun peningkatan ini relatif kecil secara numerik, fenomena tersebut mengindikasikan bahwa penerapan *penalty term* pada fungsi loss berhasil membatasi kompleksitas model dengan mengontrol besaran bobot selama proses optimisasi. Regularisasi L1 mendorong terjadinya *sparsity* pada parameter melalui penalti terhadap nilai absolut bobot, sementara regularisasi L2 melakukan *weight shrinkage* dengan meminimalkan kuadrat bobot sehingga distribusi parameter menjadi lebih terkendali. Dengan adanya pembatasan ini, model tidak lagi cenderung menyesuaikan diri secara berlebihan terhadap noise atau fluktuasi spesifik pada data latih, melainkan terdorong untuk mengekstraksi representasi fitur yang lebih stabil dan relevan,

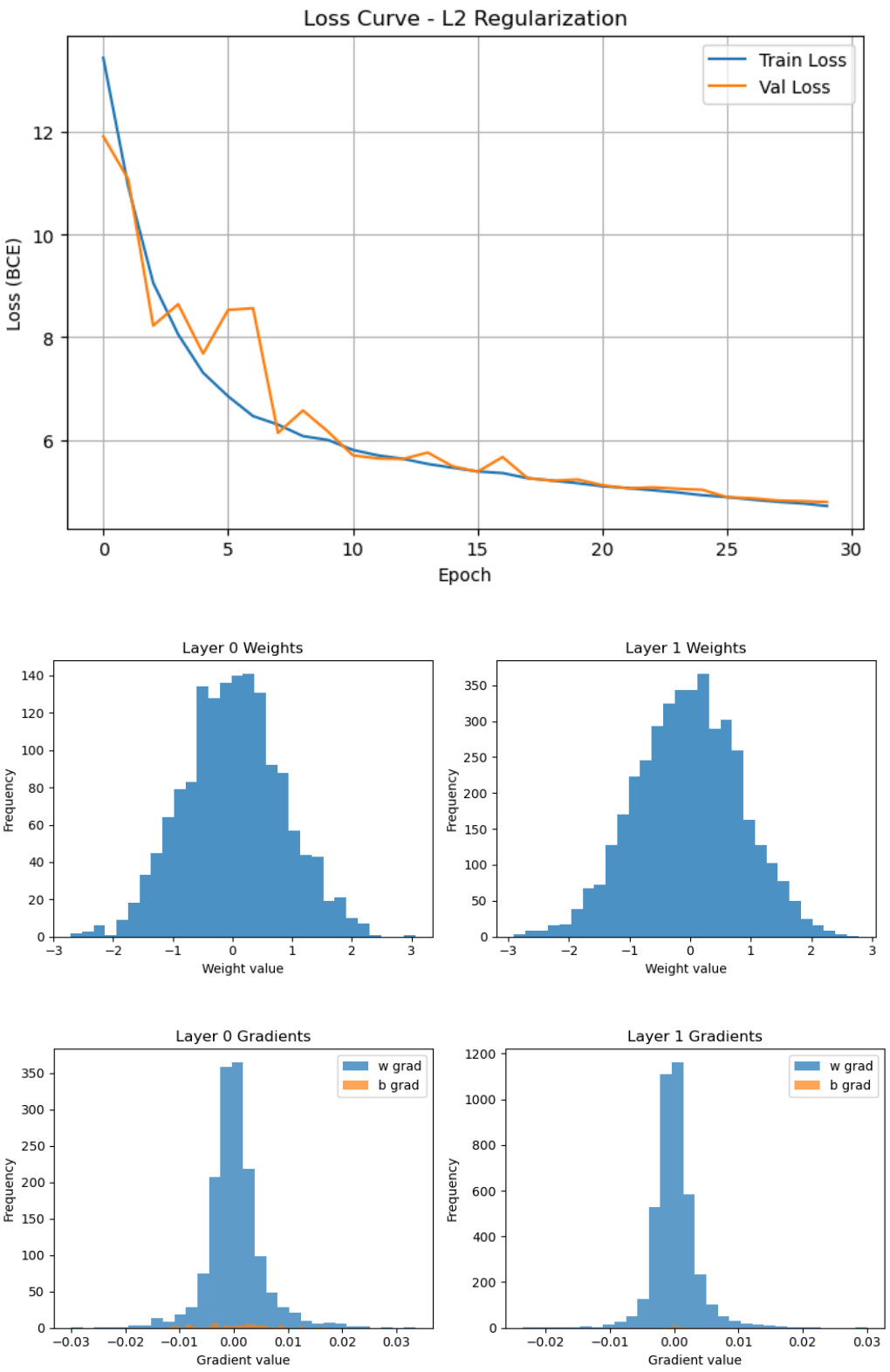
yang pada akhirnya meningkatkan kemampuan generalisasi model ketika dievaluasi pada data validasi.








Loss Curve - L1 Regularization
Train Loss
Val Loss
Loss (BCE)
Epoch
Layer 0 Weights
Layer 1 Weights
Frequency
Weight value
Layer 0 Gradients
Layer 1 Gradients
w grad
b grad
Gradient value



2.2.6. Perbandingan dengan Pustaka *scikit-learn*

Parameter	Custom FFNN	Sklearn MLPClassifier
Layers Config	[24, 64, 64, 1]	(64, 64)
Activations	['tanh', 'tanh', 'sigmoid']	tanh

Initialization	he	-
Learning Rate	0.003	0.003
Batch Size	32	32
Epoch	25	25 (Max Iter)
L2 Regularization	1e-5	1e-5
Random State	42	42

Metrik	Custom FFNN	Sklearn MLPClassifier
accuracy	0.77	0.76
Macro F1	0.76	0.74
Weighted F1	0.77	0.76
Placed (1) Precision	0.79	0.76
Placed (1) Recall	0.86	0.88
Not Placed (0) Precision	0.75	0.76
Not Placed (0) Recall	0.63	0.58

Pada pengujian perbandingan ini, arsitektur dan parameter dari Custom FFNN (Best Tuning) diselaraskan semaksimal mungkin dengan parameter Sklearn MLPClassifier agar evaluasi dapat berjalan secara *apple-to-apple*. Kedua model dilatih menggunakan struktur 2 hidden layer berisi 64 neuron (64, 64), aktivasi Tanh, solver SGD, learning rate 0.003, batch size 32, selama 25 epochs (atau max iter), serta menerapkan regularisasi L2 sebesar 1e-05.

Hasil pada Data Test menunjukkan bahwa model Custom FFNN berhasil mengungguli implementasi library Scikit-Learn. Custom FFNN meraih Akurasi sebesar 0.7747 dan Macro F1-Score sebesar 0.7561, mengalahkan Sklearn MLPClassifier yang mencatatkan Akurasi 0.7627 dan Macro F1-Score 0.7378.

Keunggulan model Custom ini terutama terlihat dari kemampuannya mengenali kelas minoritas, yaitu "Not Placed (0)". Custom FFNN mampu memberikan nilai Recall untuk kelas "Not Placed" sebesar 0.63, lebih tinggi secara signifikan dibandingkan Sklearn yang hanya mencapai 0.58. Hal ini mengindikasikan bahwa metode inisialisasi bobot (seperti He Initialization) berhasil memberikan keseimbangan yang sedikit lebih baik dalam mengatasi ketidakseimbangan kelas (*class imbalance*) pada ruang fitur, sehingga

meminimalisir kesalahan prediksi (*False Negatives*) pada kandidat yang seharusnya tidak lolos.

2.3. Analisis

Ha

BAB III
KESIMPULAN & SARAN

3.1. Kesimpulan

Berdasarkan hasil eksperimen dan pengujian yang telah dilakukan, dapat disimpulkan bahwa:

1. Menambah jumlah neuron membantu model memahami data yang kompleks dengan lebih baik. Namun, menambah jumlah lapisan (*depth*) secara berlebihan justru bisa merusak performa karena *weight update* akan sangat kecil ketika sampai ke lapisan awal (*vanishing gradient problem*)
2. ReLU adalah pilihan paling baik karena proses belajarnya cepat dan stabil. Sebaliknya, Sigmoid bekerja sangat lambat karena nilai gradiennya sering tertekan menjadi sangat kecil, sehingga model belajar lebih lambat
3. Inisialisasi bobot dengan angka nol atau acak *uniform* kurang baik karena model akan kesulitan membedakan fitur data. Metode Xavier dan He lebih baik supaya aliran informasi antarlapisan tetap stabil
4. Learning rate paling optimal adalah 0.01 karena tidak *overstep* dan tidak terlalu lambat
5. Model kami berhasil mengalahkan *library* scikit-learn dalam hal akurasi dan kemampuan mendeteksi mahasiswa yang tidak mendeteksi mahasiswa yang tidak mendapat penempatan kerja

3.2. Saran

Berdasarkan kesimpulan tersebut, beberapa saran yang dapat diberikan adalah sebagai berikut:

1. Untuk mengoptimasi proses belajar model, sebaiknya implementasikan Optimizer Adam
2. Jika menggunakan *learning rate* yang sangat kecil, jumlah *epoch* harus ditambah agar model memiliki waktu sampai ke titik paling optimal
3. Untuk membantu menyeimbangkan nilai data sebelum diproses oleh model, sebaiknya implementasikan fitur seperti RMSNorm

REFERENSI

- https://www.youtube.com/watch?v=VMj-3S1tku0
- https://builtin.com/data-science/l2-regularization
- https://benihime91.github.io/blog/machinelearning/deeplearning/python3.x/tensorflow2.x/2020/10/08/adamW.html
- https://stats.stackexchange.com/questions/275521/what-is-the-derivative-of-leaky-relu
-

LAMPIRAN

Link Repositori
https://github.com/iammadsfq/Tubes1-ML-FFNN-G60

Tabel Pembagian Tugas

NIM	BAGIAN

13523135	• engine.py
	  ○ Tensor
	  ■ Auto Differential
	  ■ Metode *dunder* operasi aritmatik (__add__, __matmul__, dll.)
	• nn.py
	  ○ Module
	  ○ Layer
	• loss.py
	  ○ MSE
	• activation.py
	  ○ Linear
	  ○ ReLU
	• notebook.ipynb
	  ○ Tuning

13523152	• nn.py
	  ○ Visualisasi bobot
	  ○ Visualisasi gradien
	  ○ Save model
	  ○ Load model
	• loss.py
	  ○ CCE
	• activation.py
	  ○ Tanh
	  ○ Softmax
	  ○ Sigmoid
	• notebook.ipynb
	  ○ Pelatihan Model
	  ○ Analisis

13523162	• engine.py
	  ○ Tensor
	  ■ Metode sum, abs untuk L1 & L2
	• nn.py
	  ○ FFNN
	  ■ fit()
	  ■ predict() (forward + backward)
	  ■ forward()

	■ Regularisasi L1 & L2 ● loss.py ○ BCE ● activation.py ○ Sigmoid ○ LeakyReLU ● optimizer.py ○ SGD

Form Penggunaan AI

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:
Nama/NIM : Ahmad Syafiq/13523135, Muhammad Kinan Arkansyaddad/13523152, Fachriza Ahmad Setyono/13523162
Fakultas/Sekolah : STEI/Teknik Informatika
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : Feedforward Neural Network

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	-
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	3
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan:		**-**

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 18 Maret 2026

Ahmad Syafiq

Kode Sumber

engine.py

```
import numpy as np

class Tensor:
    """ Menyimpan nilai skalar/matriks dan gradiennya untuk Autodiff. """
    def __init__(self, data, _children=(), _op=''):
        self.data = np.array(data)
        self.grad = np.zeros_like(self.data)
        self._prev = set(_children)
        self._backward = lambda: None

    def __add__(self, other):
        other = other if isinstance(other, Tensor) else Tensor(other)
        out = Tensor(self.data + other.data, (self, other), '+')

        def _backward():
            grad_self = out.grad
            grad_other = out.grad

            while grad_self.ndim > self.data.ndim:
                grad_self = np.sum(grad_self, axis=0)
            for i in range(self.data.ndim):
                if self.data.shape[i] == 1 and grad_self.shape[i] > 1:
                    grad_self = np.sum(grad_self, axis=i, keepdims=True)

            while grad_other.ndim > other.data.ndim:
                grad_other = np.sum(grad_other, axis=0)
            for i in range(other.data.ndim):
                if other.data.shape[i] == 1 and grad_other.shape[i] > 1:
                    grad_other = np.sum(grad_other, axis=i, keepdims=True)

            self.grad += grad_self
            other.grad += grad_other

        out._backward = _backward
        return out

    def __matmul__(self, other):
        out = Tensor(self.data @ other.data, (self, other), '@')

        def _backward():
            self.grad += out.grad @ other.data.T
            other.grad += self.data.T @ out.grad

        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Tensor) else Tensor(other)
        out = Tensor(self.data * other.data, (self, other), '*')

        def _backward():
            grad_self = other.data * out.grad
```

```
            grad_other = self.data * out.grad

            while grad_self.ndim > self.data.ndim:
                grad_self = np.sum(grad_self, axis=0)
            for i in range (self.data.ndim):
                if self.data.shape[i] == 1 and grad_self.shape[i] > 1:
                    grad_self = np.sum(grad_self, axis=i, keepdims=True)

            while grad_other.ndim > other.data.ndim:
                grad_other = np.sum(grad_other, axis=0)
            for i in range(other.data.ndim):
                if other.data.shape[i] == 1 and grad_other.shape[i] > 1:
                    grad_other = np.sum(grad_other, axis=i, keepdims=True)

            self.grad += grad_self
            other.grad += grad_other

        out._backward = _backward
        return out

    def __neg__(self):
        return self*-1

    def __sub__(self,other):
        return self + (-other)

    def __pow__(self, n):
        assert isinstance(n, (int,float)), "n hanya bisa angka biasa"
        out = Tensor(self.data**n, (self,), f'**{n}')

        def _backward():
            self.grad += (n*self.data**(n-1))*out.grad

        out._backward = _backward
        return out

    def sum(self):
        out = Tensor(np.sum(self.data), (self,), 'sum')

        def _backward():
            self.grad += np.ones_like(self.data) * out.grad

        out._backward = _backward
        return out

    def abs(self):
        out = Tensor(np.abs(self.data), (self,), 'abs')

        def _backward():
            # deriv of |x| is sign of x
            self.grad += np.sign(self.data) * out.grad

        out._backward = _backward
        return out

    def backward(self):
        """ Memulai proses backpropagation dari node ini. """
        self.grad = np.ones_like(self.data)

        topo = []
        visited = set()

        def build_topo(v):
            if v not in visited:
                visited.add(v)
                for child in v._prev:
                    build_topo(child)
```

```
                topo.append(v)

        build_topo(self)

        for node in reversed(topo):
            node._backward()
```

nn.py

```
iimport os
import pickle
from .engine import Tensor
from models.optimizers import *
import models.activations as activations
import models.loss as loss_module
import numpy as np
import sys
import matplotlib.pyplot as plt

class Module:
    def zero_grad(self):
        for p in self.parameters():
            p.grad = np.zeros_like(p.grad)

    def parameters(self):
        return []

class Layer(Module):
    def __init__(
            self,
            nin,
            nout,
            activation_name='linear',
            init_method='normal',
            random_state=42,
            **kwargs):

        if random_state is not None:
            np.random.seed(random_state)

        self.activation_name = activation_name.lower()


        if not isinstance(init_method, str):
            raise ValueError("init_method must be str")

        init_method = init_method.lower()
```

```
        if init_method not in {'zero', 'uniform', 'normal', 'xavier', 'he'}:
            raise ValueError(f"activation name not supported {init_method}")

        if init_method == 'zero':
            w_data = np.zeros((nin, nout))
        elif init_method == 'uniform':
            low, high = kwargs.get('lower', -0.1), kwargs.get('upper', 0.1)
            w_data = np.random.uniform(low, high, (nin, nout))
        elif init_method == 'normal':
            mean, var = kwargs.get('mean', 0.0), kwargs.get('variance', 1.0)
            w_data = np.random.normal(mean, np.sqrt(var), (nin,nout))
        elif init_method == 'xavier':
            w_data = np.random.randn(nin, nout) * np.sqrt(1/nin)
        elif init_method == 'he':
            w_data = np.random.randn(nin, nout) * np.sqrt(2/nin)

        self.random_state = random_state
        self.w = Tensor(w_data)
        self.b = Tensor(np.zeros((1, nout)))

    @property
    def activation_name(self):
        return self._activation_name

    @activation_name.setter
    def activation_name(self, value):
        if not isinstance(value, str):
            raise ValueError("activation must be str")

        if value not in {'linear', 'relu', 'sigmoid', 'tanh', 'softmax',
'leakyrelu'}:
            raise ValueError(f"activation name not supported {value}")

        self._activation_name = value

        if self.activation_name == 'linear':
            self.activation = activations.Linear()
        elif self.activation_name == 'relu':
            self.activation = activations.ReLU()
        elif self.activation_name == 'sigmoid':
            self.activation = activations.Sigmoid()
        elif self.activation_name == 'tanh':
            self.activation = activations.TanH()
        elif self.activation_name == 'softmax':
            self.activation = activations.Softmax()
        elif self.activation_name == 'leakyrelu':
            self.activation = activations.LeakyReLU()
```

```
    def __call__(self, x):
        return self.activation(x @ self.w + self.b)

    def parameters(self):
        return [self.w, self.b]

class FFNN(Module):
    def __init__(
        self,
        layers_config,
        activations='relu', # bisa string atau array of strings
        loss_function='mse',
        init_method='normal',
        random_state=42
    ):
        self.layers_config = layers_config
        self.loss_name = loss_function
        self.init_method = init_method
        self.random_state = random_state
        self.layers = []

        if isinstance(activations, str):
            activations = [activations] * (len(layers_config) - 1)

        for i in range(len(layers_config) - 1):
            nin = layers_config[i]
            nout = layers_config[i+1]
            act = activations[i]
            layer_seed = random_state + i if random_state is not None else None
            self.layers.append(Layer(nin, nout, activation_name=act,
init_method=init_method, random_state=layer_seed))

    @property
    def loss_name(self):
        return self._loss_name

    @loss_name.setter
    def loss_name(self, value):
        if not isinstance(value, str):
            raise ValueError("loss_name must be str")

        value = value.lower()

        if value not in {'mse', 'mseloss', 'bce', 'binaryclassentropy', 'cce',
'categoricalclassentropy'}:
            raise ValueError(f"loss name not supported {value}")
```

```
        self._loss_name = value

        if self.loss_name in {'mse', 'mseloss'}:
            self.loss_fn = loss_module.MSELoss()
        elif self.loss_name in {'binaryclassentropy', "bce"}:
            self.loss_fn = loss_module.BinaryClassEntropy()
        elif self.loss_name in {'categoricalclassentropy', 'cce'}:
            self.loss_fn = loss_module.CategoricalClassEntropy()

    def parameters(self):
        params = []
        for layer in self.layers:
            params.extend(layer.parameters())
        return params

    def fit(self,
            X_train,
            y_train,
            X_val=None,
            y_val=None,
            solver='sgd',
            batch_size=32,
            lr=0.01,
            l1_lambda=0.0,
            l2_lambda=0.0,
            epochs=100,
            verbose=1
            ):

        # [x] forward pass
        # [x] calc loss
        # [x] regularisation (L1/L2)
        # [x] backward pass
        # [x] optimizer step
        # [x] append loss history
        # [x] progress bar

        if solver.lower() == 'sgd':
            optimizer: BaseOptimizer = SGD(self.parameters(), lr=lr)
        elif solver.lower() == 'adam':
            optimizer: BaseOptimizer = Adam()
        else:
            raise ValueError(f"{solver} opitimzer is not supported yet.")

        history = {'train_loss': [], 'val_loss': []}
        n_samples = X_train.shape[0]
```

```
        n_batches = int(np.ceil(n_samples / batch_size)) # for prog bar

        for epoch in range(epochs):
            indices = np.arange(n_samples)
            np.random.shuffle(indices)

            epoch_loss = 0.0

            if verbose:
                print(f"Epoch {epoch+1}/{epochs}")

            # train
            for i in range(0, n_samples, batch_size):
                batch_indices = indices[i:i+batch_size]
                X_batch = Tensor(X_train[batch_indices])
                y_batch = Tensor(y_train[batch_indices])

                y_pred = self.forward(X_batch)

                base_loss = self.loss_fn(y_batch, y_pred)
                reg_loss = Tensor(0.0)
                for layer in self.layers:
                    w = layer.w # gk usah bias
                    if l1_lambda > 0.0:
                        reg_loss += w.abs().sum() * l1_lambda
                    if l2_lambda > 0.0:
                        reg_loss += (w ** 2).sum() * l2_lambda

                total_loss = base_loss + reg_loss
                epoch_loss += total_loss.data

                self.zero_grad()
                total_loss.backward()

                optimizer.step()

                if verbose:
                    current_batch = (i // batch_size) + 1
                    progress = current_batch / n_batches
                    bar_length = 30
                    filled_length = int(bar_length * progress)

                    # if filled_length == bar_length:
                    bar = '[\033[32m' + '-' * filled_length + '>' *
(filled_length != bar_length) + '\033[0m' + '.' * (bar_length - filled_length -
1) + ']'
                    # else:
```

```
                    #     bar = '-' * filled_length + '>' + '.' * (bar_length -
filled_length - 1)

                    current_avg_loss = epoch_loss / current_batch

                    sys.stdout.write(f"\r{current_batch}/{n_batches} {bar} -
loss: {current_avg_loss:.4f}")
                    sys.stdout.flush()

            avg_train_loss = epoch_loss / n_batches
            history['train_loss'].append(avg_train_loss)

            # validation
            val_loss_str = ""
            if X_val is not None and y_val is not None:
                X_val_t = Tensor(X_val)
                y_val_t = Tensor(y_val)

                y_val_pred = self.forward(X_val_t)
                val_base_loss = self.loss_fn(y_val_t, y_val_pred)

                val_reg_loss = Tensor(0.0)
                for layer in self.layers:
                    w = layer.w
                    if l1_lambda > 0.0:
                        val_reg_loss = val_reg_loss + w.abs().sum() * l1_lambda
                    if l2_lambda > 0.0:
                        val_reg_loss = val_reg_loss + (w ** 2).sum() * l2_lambda

                total_val_loss = val_base_loss + val_reg_loss
                avg_val_loss = total_val_loss.data

                history['val_loss'].append(avg_val_loss)
                val_loss_str = f" - val_loss: {avg_val_loss:.4f}"

            if verbose:
                sys.stdout.write(val_loss_str + "\n")


        return history

    def predict(self, X):
        out = self.forward(Tensor(X))
        return out.data

    def forward(self, x):
        out = x
```

```
        for layer in self.layers:
            out = layer(out)
        return out

    def save(self, path):
        os.makedirs(os.path.dirname(path), exist_ok=True)
        state = {
            "rand": self.random_state,
            "layers_config": self.layers_config,
            "loss_name": self.loss_name,
            "activations": [layer.activation_name for layer in self.layers],
            "init_method": self.init_method,
            "params": [
                {"rand": layer.random_state, "w": layer.w.data, "b":
layer.b.data} for layer in self.layers
            ],
        }
        with open(path, "wb") as f:
            pickle.dump(state, f)

    def load(self, path):
        with open(path, "rb") as f:
            state = pickle.load(f)

        if len(state["params"]) != len(self.layers):
            raise ValueError(
                f"Loaded model has {len(state['params'])} layers, but current
model has {len(self.layers)}."
            )

        for layer, layer_activation_name, layer_state in zip(self.layers,
state["activations"], state["params"]):
            if layer.w.data.shape != layer_state["w"].shape:
                raise ValueError(
                    f"Shape mismatch for weights: {layer.w.data.shape} vs
{layer_state['w'].shape}"
                )
            if layer.b.data.shape != layer_state["b"].shape:
                raise ValueError(
                    f"Shape mismatch for bias: {layer.b.data.shape} vs
{layer_state['b'].shape}"
                )
            layer.activation_name = layer_activation_name
            layer.random_state = layer_state["rand"]
            layer.w.data = np.array(layer_state["w"], dtype=layer.w.data.dtype)
            layer.b.data = np.array(layer_state["b"], dtype=layer.b.data.dtype)
            layer.w.grad = np.zeros_like(layer.w.data)
```

```
            layer.b.grad = np.zeros_like(layer.b.data)

        self.loss_name = state.get("loss_name", self.loss_name)
        self.init_method = state.get("init_method", self.init_method)
        self.random_state = state.get("rand", self.loss_name)
        print(f"Model successfully loaded from {path}")

    @classmethod
    def load_new(cls, path):
        """
        Load and create new instance
        """
        with open(path, "rb") as f:
            state = pickle.load(f)

        instance = cls(
            state['layers_config'],
            state['activations'],
            state['loss_name'],
            state['init_method'],
            state['rand']
        )

        for layer, layer_activation_name, layer_state in zip(instance.layers,
state["activations"], state["params"]):
            if layer.w.data.shape != layer_state["w"].shape:
                raise ValueError(
                    f"Shape mismatch for weights: {layer.w.data.shape} vs
{layer_state['w'].shape}"
                )
            if layer.b.data.shape != layer_state["b"].shape:
                raise ValueError(
                    f"Shape mismatch for bias: {layer.b.data.shape} vs
{layer_state['b'].shape}"
                )
            layer.activation_name = layer_activation_name
            layer.random_state = layer_state["rand"]
            layer.w.data = np.array(layer_state["w"], dtype=layer.w.data.dtype)
            layer.b.data = np.array(layer_state["b"], dtype=layer.b.data.dtype)
            layer.w.grad = np.zeros_like(layer.w.data)
            layer.b.grad = np.zeros_like(layer.b.data)

        return instance
```

```python
    def plot_weights_distribution(self, layer_indices, bins = 30):
        if layer_indices is None:
            layer_indices = list(range(len(self.layers)))
        if isinstance(layer_indices, int):
            layer_indices = [layer_indices]

        plt.figure(figsize=(5 * len(layer_indices), 4))
        for idx, layer_idx in enumerate(layer_indices):
            if layer_idx < 0 or layer_idx >= len(self.layers):
                raise IndexError(f"Layer index {layer_idx} out of range.")

            w = self.layers[layer_idx].w.data.flatten()
            plt.subplot(1, len(layer_indices), idx + 1)
            plt.hist(w, bins=bins, color="C0", alpha=0.8)
            plt.title(f"Layer {layer_idx} Weights")
            plt.xlabel("Weight value")
            plt.ylabel("Frequency")
        plt.tight_layout()
        plt.show()

    def plot_gradients_distribution(self, layer_indices, bins = 30):
        if layer_indices is None:
            layer_indices = list(range(len(self.layers)))
        if isinstance(layer_indices, int):
            layer_indices = [layer_indices]

        plt.figure(figsize=(5 * len(layer_indices), 4))
        for idx, layer_idx in enumerate(layer_indices):
            if layer_idx < 0 or layer_idx >= len(self.layers):
                raise IndexError(f"Layer index {layer_idx} out of range.")

            w_grad = self.layers[layer_idx].w.grad.flatten()
            b_grad = self.layers[layer_idx].b.grad.flatten()

            plt.subplot(1, len(layer_indices), idx + 1)
            if w_grad.size > 0:
                plt.hist(w_grad, bins=bins, alpha=0.7, label="w grad")
            if b_grad.size > 0:
                plt.hist(b_grad, bins=bins, alpha=0.7, label="b grad")
            plt.title(f"Layer {layer_idx} Gradients")
            plt.xlabel("Gradient value")
            plt.ylabel("Frequency")
            plt.legend()
        plt.tight_layout()
        plt.show()
```

activations.py

```
import numpy as np
from models.engine import Tensor
from abc import ABC, abstractmethod

class BaseActivation(ABC):
    """Do FF by calling the instance"""
    @abstractmethod
    def forward(self, x):
        raise NotImplementedError

    def __call__(self, x):
        return self.forward(x)

class Linear(BaseActivation):
    """Linear(x) = x"""
    def forward(self, x):
        out = Tensor(x.data, (x,), 'linear')

        def _backward():
            x.grad += 1.0*out.grad

        out._backward = _backward
        return out

class ReLU(BaseActivation):
    """ReLU(x) = max(0, x)"""
    def forward(self, x):
        out = Tensor(np.maximum(0, x.data), (x,), 'ReLU')

        def _backward():
            x.grad += (x.data > 0).astype(float)*out.grad # 1 kalo x > 0; else 0

        out._backward = _backward
        return out

class Sigmoid(BaseActivation):
    """Sigmoid(x) = 1/(1+exp(-x))"""
    def forward(self, x):

        sig = np.where(x.data >= 0, 1 / (1 + np.exp(-x.data)), np.exp(x.data)/ (1
+ np.exp(x.data)))

        out = Tensor(sig, (x,), 'Sigmoid')
        def _backward():
```

```
            x.grad += (sig * (1.0 - sig)) * out.grad

        out._backward = _backward
        return out

class TanH(BaseActivation):
    """tanh(x) = (exp(x) - exp(-x))/(exp(x) + exp(-x))"""
    def forward(self, x):
        e1 = np.exp(x.data)
        e2 = np.exp(-x.data)

        v = (e1-e2)/(e1+e2)
        out = Tensor(v, (x,), 'Tanh')

        def _backward():
            x.grad += (1.0 - v**2) * out.grad

        out._backward = _backward
        return out

class Softmax(BaseActivation):
    """
    Softmax(x)_i = exp(x_i) / sum(exp(x_j))
    Dapat menerima input berupa batch
    Refrensi:
https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/
    """
    def forward(self, x):
        x_shifted = x.data - np.max(x.data, axis=1, keepdims=True) #shifted untuk
numerical stability
        exp_x = np.exp(x_shifted)
        sm = exp_x / np.sum(exp_x, axis=1, keepdims=True)
        out = Tensor(sm, (x,), 'Softmax')
        def _backward():
            grad = np.zeros_like(sm)
            for i in range(len(sm)):
                s_i = np.reshape(sm[i], (-1, 1))
                jm = np.diagflat(s_i) - (np.dot(s_i, s_i.T))
                grad[i] = np.dot(jm, out.grad[i])
            x.grad += grad

        out._backward = _backward
        return out


## BONUS
```

```
class LeakyReLU(BaseActivation):
    """Leaky ReLU(x) = ax if x <= 0 else x"""
    def __init__(self, a=0.01):
        super().__init__()
        self.a = a

    def forward(self, x):
        out = Tensor( np.where(x.data <= 0, self.a*x.data, x.data), (x,), 'Leaky
ReLU')

        def _backward():
            x.grad += np.where(x.data <= 0, self.a, 1) * out.grad

        out._backward = _backward
        return out
```

loss.py

```
class BinaryClassEntropy(BaseLoss):
    """
    Menghitung Binary Cross-Entropy untuk klasifikasi biner
    Menggunakan ln
    """
    def forward(self, y_true: Tensor, y_pred: Tensor) -> Tensor:
        assert y_true.data.shape == y_pred.data.shape, \
            f"Shape mismatch: y_true {y_true.data.shape} vs y_pred
{y_pred.data.shape}"

        n = y_true.data.shape[0] # batch size

        # avoid log(0)
        eps = 1e-15
        y_pred_clipped = np.clip(y_pred.data, eps, 1 - eps)

        loss_val = -np.mean(y_true.data * np.log(y_pred_clipped) + (1 -
y_true.data) * np.log(1 - y_pred_clipped))

        out = Tensor(loss_val, (y_pred,), 'BCE')

        def _backward():
            grad = (y_pred_clipped - y_true.data) / (y_pred_clipped * (1.0 -
y_pred_clipped))
            y_pred.grad += (1.0 / n) * grad * out.grad

        out._backward = _backward
        return out
```

```
class CategoricalClassEntropy(BaseLoss):
    """
    Menghitung Categorical Cross-Entropy untuk multi-class
    y_true diasumsikan dalam bentuk one-hot encoding
    """
    def forward(self, y_true: Tensor, y_pred: Tensor) -> Tensor:
        assert y_true.data.shape == y_pred.data.shape, \
            f"Shape mismatch: y_true {y_true.data.shape} vs y_pred
{y_pred.data.shape}"

        n = y_true.data.shape[0] # Batch size

        # avoid log(0)
        eps = 1e-15
        y_pred_clipped = np.clip(y_pred.data, eps, 1 - eps)

        loss_val = -np.sum(y_true.data * np.log(y_pred_clipped)) / n

        out = Tensor(loss_val, (y_pred,), 'CCE')

        def _backward():
            grad = -(y_true.data / y_pred_clipped)
            y_pred.grad += (1.0/n) * grad * out.grad

        out._backward = _backward
        return out
```

optimizers.py

```
from abc import ABC, abstractmethod

# aku cinta abstraksi
class BaseOptimizer(ABC):
     @abstractmethod
     def step(self) -> None:
          pass

class SGD(BaseOptimizer):
    def __init__(self, parameters, lr):
        self.parameters = parameters
        self.lr = lr

    def step(self):
        for p in self.parameters:
                p.data -= self.lr * p.grad
```

```python
class Adam(BaseOptimizer): # ini kalo mw implement adam

    # mas adam~ mas adam~
    # btw implement Adam ya, jangan AdamW

    def step(self) -> None:
        pass
```